

Plan Topology Schema Reference

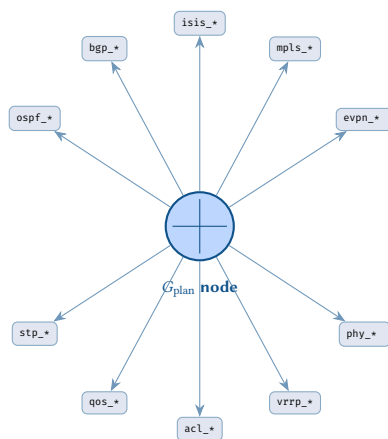
Attribute Catalogue, Design Patterns, and Validation Rules

Simon Knight, Ph.D.

Adelaide, Australia

March 2026 • Version 1.0.0

Last updated: March 18, 2026



Abstract. This document is the practitioner's reference companion to *A Vendor-Neutral Multi-Layer Graph Model for Network Protocol State* (ANK-TR-2026-03). It provides a complete attribute catalogue for the plan topology G_{plan} , covering every protocol family from physical-layer optics through BGP, EVPN, and security policy. The document defines the naming conventions and type system for flat attributes, presents a library of reusable design patterns with TikZ diagrams and attribute tables, catalogues all validation rules with severity levels, and walks through two end-to-end worked examples. A migration guide maps vendor CLI constructs to the flat attribute schema. Engineers implementing or extending the plan topology should use this document as their primary day-to-day reference.

Contents

1	Introduction	1
1.1	Purpose	1
1.2	Scope	2
1.3	How to Read This Document	2
1.4	Cross-Reference to the Main Report	2
2	Attribute Namespace	2
2.1	Naming Conventions	2
2.2	Type System	3
2.3	Attribute Lifecycle	4
3	Complete Attribute Catalogue	5
3.1	Device Attributes (Non-Protocol)	5
3.2	Interface/Endpoint Attributes (Non-Protocol)	5
3.3	IP Addressing (ip_*)	6
3.4	OSPF (ospf_*)	6
3.5	IS-IS (isis_*)	7
3.6	BGP (bgp_*)	8
3.7	MPLS and Segment Routing (mpls_*, sr_*)	8
3.8	EVPN (evpn_*)	9
3.9	STP (stp_*)	9
3.10	L2 / Switching (switchport_*, vlan_*, lacp_*)	10
3.11	VRRP/HSRP (vrrp_*, hsrp_*)	10
3.12	QoS (qos_*)	11
3.13	ACL (acl_*)	11
3.14	Security (sgt_*, macsec_*, dot1x_*)	11
3.15	Physical Layer (phy_*)	12
4	Design Rules: Primitives and Assertions	12
5	Design Pattern Catalogue	13
5.1	BGP Patterns	13
5.2	IGP Patterns	16
5.3	MPLS/SR Patterns	18
5.4	EVPN-VXLAN Patterns	19
5.5	Campus/Access Patterns	20
5.6	Security Patterns	22
5.7	Composite Patterns	23
6	Validation Rule Reference	24
6.1	Structural Constraints	25
6.2	Per-Protocol Consistency Rules	25
6.3	Cross-Protocol Consistency Rules	27
6.4	Completeness Rules	28
7	Worked Examples	29
7.1	Small Enterprise Network	29
7.2	DC Leaf-Spine Fabric	30
8	Migration Guide	32
8.1	Populating G_{plan} from Existing Configs	32
8.2	Cisco IOS/IOS-XE Mapping	32
8.3	Juniper Junos Mapping	32
8.4	Arista EOS Mapping	32
8.5	Handling Vendor-Specific Features	32

1 Introduction

1.1 Purpose

This document is the practitioner’s reference for the plan topology G_{plan} defined in the companion technical report ANK-TR-2026-03 [1]. While the main report establishes the formal mathematical foundations—the labelled property hypergraph, functorial overlay mappings, and correctness guarantees—this reference focuses

on the concrete schema that engineers work with daily: what attributes exist, what types they carry, how they are named, and how they compose into reusable patterns.

Every attribute listed in this document maps directly to a key in the property map π_V (for node-scoped attributes) or π_E (for endpoint-scoped attributes) of G_{plan} . The functor $\mathcal{F} : G_{\text{plan}} \rightarrow G_{\text{overlay}}$ reads these attributes to construct the protocol-specific overlay graphs described in the main report.

1.2 Scope

This document covers four areas:

1. **Attribute schema** — the naming grammar, type system, and null semantics that govern every attribute key and value in G_{plan} .
2. **Attribute catalogue** — the complete per-protocol listing of every defined attribute, its type, scope, optionality, default value, and constraints.
3. **Design patterns** — reusable topology fragments with pre-populated attribute tables that serve as building blocks for real network designs.
4. **Validation rules** — the full set of structural, per-protocol, and cross-protocol consistency checks that the validation pipeline enforces before overlay generation.

1.3 How to Read This Document

If you are implementing a new protocol functor, start with §2 to understand the naming and type conventions, then consult the relevant protocol subsection in §3 for the attribute table.

If you are designing a network, browse the design patterns in §5 to find a topology fragment that matches your requirements, then use the attribute catalogue to fill in the details.

If your plan topology fails validation, consult the validation rule reference in §6 to understand the constraint that was violated and how to fix it.

If you are migrating existing configs into G_{plan} , start with the migration guide in §8 for vendor-specific CLI mappings.

1.4 Cross-Reference to the Main Report

The formal definitions of G_{plan} , the functor algebra $\mathcal{F}$, and the validation pipeline appear in ANK-TR-2026-03 [1]. In particular:

- The plan topology structure is defined in Design Rules 1–3 of the main report.
- The collection flattening rules are formalised in the main report’s §3 and summarised here in §2.2.
- The functor definitions ($\mathcal{F}_{\text{OSPF}}$, $\mathcal{F}_{\text{BGP}}$, etc.) appear in the main report’s Part III; this document lists the input attributes each functor reads.

2 Attribute Namespace

2.1 Naming Conventions

Every attribute key in G_{plan} follows a strict naming grammar. This grammar ensures that attributes are self-documenting, collision-free across protocols, and mechanically parseable by the functor pipeline.

Design Rule 1: Attribute Key Grammar

All attribute keys conform to the following EBNF grammar:

```
attr_key      = protocol_prefix "_" field_name
               | device_field ;
protocol_prefix = "ospf" | "bgp" | "isis" | "stp"
               | "mpls" | "sr" | "evpn" | "vrrp"
               | "hsrp" | "acl" | "qos" | "sgt"
               | "macsec" | "dot1x" | "ip"
               | "switchport" | "vlan" | "lacp"
               | "lldp" | "phy" | "dom"
               | "rsvp" | "ldp" | "bfd" ;
field_name    = identifier { "_" identifier } ;
identifier    = letter { letter | digit } ;
device_field  = "hostname" | "role" | "platform"
               | "os_version" | "management_ip"
               | "name" | "description"
               | "admin_status" | "mtu" | "speed"
               | "duplex" ;
indexed_key   = attr_key "[" index "]" ;
```

```
index          = digit { digit } ;
```

2.1.1 Rules for Indexed Attributes

Some protocols require ordered sequences of sub-attributes. ACL rule entries and static route tables are the primary examples. Indexed attributes use a bracket suffix:

- `acl_MGMT_seq_10_action` — the action for sequence number 10 of ACL “MGMT”.
- `acl_MGMT_seq_10_protocol` — the protocol match for the same rule.
- `secondary_ipv4[0]`, `secondary_ipv4[1]` — secondary IPv4 addresses on an interface, zero-indexed.

The index in brackets is always a non-negative integer. For ACLs, the index corresponds to the sequence number used in the running configuration. For other indexed attributes, indices start at zero and are contiguous.

2.1.2 Rules for Per-AFI Attributes

BGP address-family attributes use an AFI/SAFI infix between the protocol prefix and the field name:

- `bgp_ipv4_unicast` — IPv4 unicast address family enabled.
- `bgp_ipv6_unicast` — IPv6 unicast address family enabled.
- `bgp_l2vpn_evpn` — L2VPN EVPN address family enabled.

This pattern generalises to any AFI/SAFI combination. The infix always appears immediately after `bgp_` and before the field-specific suffix.

2.1.3 Rules for Per-VLAN Attributes

STP and other VLAN-aware protocols embed the VLAN ID in the attribute key:

- `stp_vlan_100_priority` — STP priority for VLAN 100.
- `stp_vlan_200_priority` — STP priority for VLAN 200.

The VLAN ID is always a decimal integer in the range 1–4094. When a per-VLAN attribute is absent, the global default (e.g., `stp_priority`) applies.

2.2 Type System

2.2.1 Primitive Types

Every attribute value belongs to exactly one of the following primitive types:

Table 1. Primitive types in the plan topology attribute schema.

Type	Representation	Examples
bool	Boolean true/false	true, false
u32	Unsigned 32-bit integer	1500, 65001, 32768
u64	Unsigned 64-bit integer	4294967296 (large label values)
f64	64-bit floating point	100.0 (reference bandwidth in Gbps)
string	UTF-8 string	"spine-01", "Loopback0"
ipv4_addr	IPv4 address (no prefix)	10.0.0.1
ipv6_addr	IPv6 address (no prefix)	2001:db8::1
ip_prefix	IP address with prefix length	10.0.1.0/31, 2001:db8::/48
mac_addr	IEEE 802 MAC address	00:11:22:33:44:55
enum	Named enumeration	up, point-to-point, ibgp

2.2.2 Collection Flattening Rules

The plan topology stores all state as flat key-value pairs. Structured or nested data from device configurations must be flattened according to the rules established in the main report [1]:

1. **Ordered lists** are flattened to indexed attributes: a list [A, B, C] at key k becomes `k[0]=A`, `k[1]=B`, `k[2]=C`.
2. **Sets** (e.g., allowed VLANs) are flattened to a comma-separated string in a single attribute: `allowed_vlans="100,200,300"`. The functor parses this string into the native set type during overlay construction.
3. **Maps** (e.g., per-neighbour BGP policies) are encoded by embedding the map key in the attribute name: `bgp_neighbor_10.0.0.2_route_map_in="RM-PEER"`. However, in practice, per-neighbour state is modelled as endpoint attributes on the link connecting the two BGP speakers, avoiding the need for map flattening in most cases.

Flattening an ACL

Consider a simple ACL with two rules:

```
ip access-list extended MGMT
 10 permit tcp 10.0.0.0/24 any eq 22
 20 deny ip any any
```

This flattens to the following node-level attributes:

```
acl_MGMT_seq_10_action = "permit"
acl_MGMT_seq_10_protocol = "tcp"
acl_MGMT_seq_10_src = "10.0.0.0/24"
acl_MGMT_seq_10_dst = "any"
acl_MGMT_seq_10_dst_port = "22"
acl_MGMT_seq_20_action = "deny"
acl_MGMT_seq_20_protocol = "ip"
acl_MGMT_seq_20_src = "any"
acl_MGMT_seq_20_dst = "any"
```

Figure 1 illustrates these flattening rules for the three collection types.

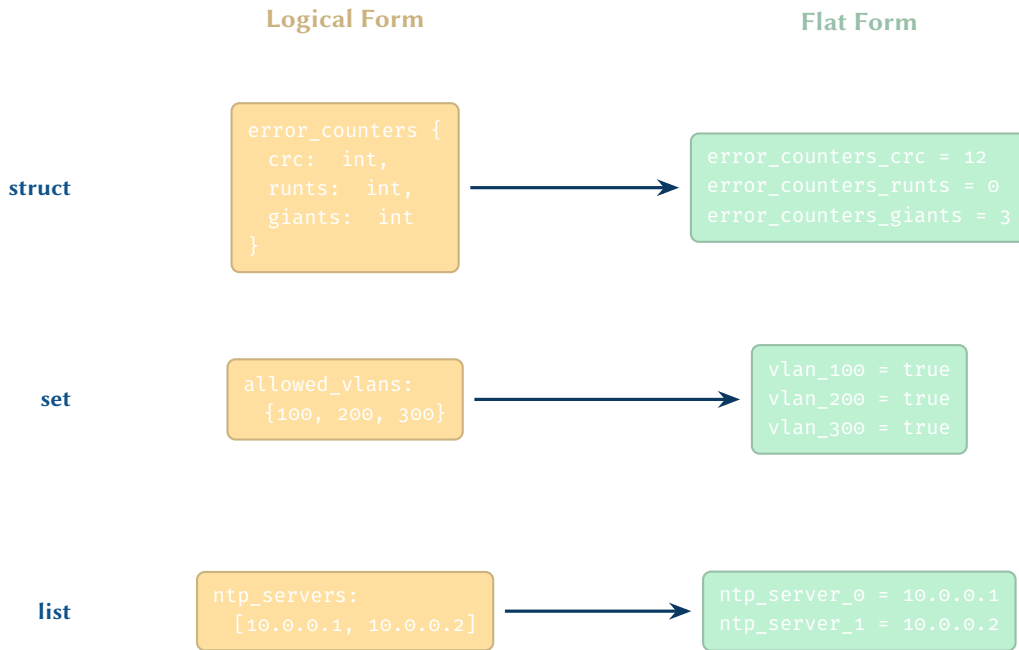


Figure 1. Flattening conventions: struct, set, and list types mapped to flat key-value attributes.

2.2.3 Null Semantics

An attribute that is *absent* from a node or endpoint means that the corresponding protocol or feature is not configured on that element. This is distinct from an attribute being *present with a default value*.

- If `ospf_enabled` is absent from a node, the OSPF functor $\mathcal{F}_{\text{OSPF}}$ skips that node entirely.
- If `ospf_cost` is absent from an endpoint but `ospf_area` is present, the functor uses the default cost (derived from `ospf_reference_bandwidth` and speed).
- If `ospf_area` is absent from an endpoint on an OSPF-enabled node, the validation pipeline raises a warning (the interface is not participating in OSPF).

This convention means the attribute map is *sparse*: a device running only OSPF and BGP has no STP, EVPN, or MPLS attributes. The functor pipeline naturally handles this by checking for the `<protocol>_enabled` gate attribute before reading any protocol-specific keys.

2.3 Attribute Lifecycle

Every attribute in G_{plan} passes through a well-defined lifecycle. An attribute is initially *absent*, meaning the corresponding feature is not configured. When an engineer populates the plan topology, the attribute transitions to the *authored* state. Required attributes then enter the validation pipeline, which checks type constraints, cross-endpoint consistency, and protocol-specific rules. Attributes that pass validation become *valid* and are available for functor consumption. Figure 2 traces this progression, including the error and warning branches for failed or missing attributes.

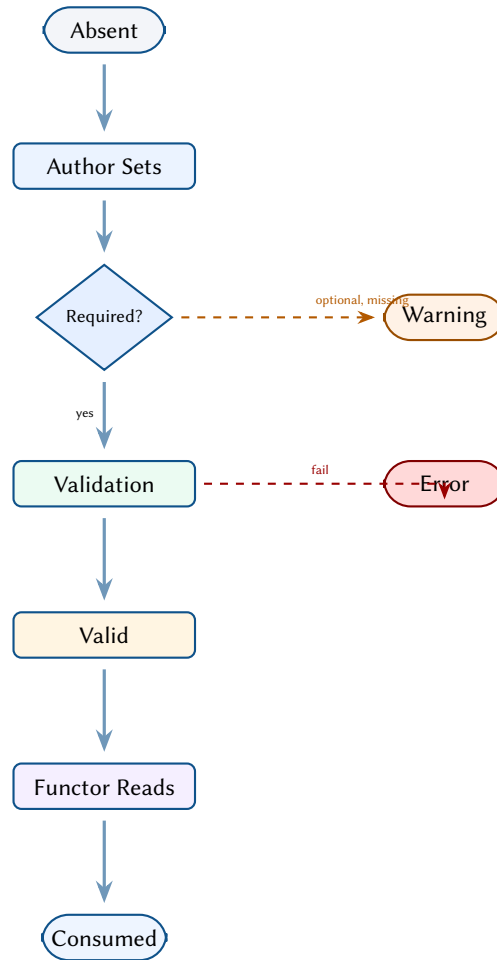


Figure 2. Attribute lifecycle in G_{plan} : from absent through authoring, validation, and functor consumption. Required attributes that fail validation produce errors (red); optional attributes that are absent produce warnings (orange).

3 Complete Attribute Catalogue

This section provides the exhaustive listing of every defined attribute in the plan topology schema, organised by protocol family. Each subsection contains a table with the following columns:

- **Attribute Key** – the flat key string.
- **Type** – one of the primitive types from Table 1.
- **Scope** – whether the attribute is stored on the *node* (π_V) or on an *endpoint* (π_E).
- **Req/Opt** – whether the attribute is required (R) when the protocol is enabled, or optional (O).
- **Default** – the default value when the attribute is absent but the protocol is active; “–” means no default (must be explicitly set if required).
- **Description** – a brief explanation.
- **Constraints** – value range or format restrictions.

The subsections below are ordered by protocol dependency: device and interface attributes come first (read by every functor), followed by IP, IGP, BGP, MPLS/SR, EVPN, L2, and policy layers. Each subsection corresponds to a prefix in the naming grammar from §2.1. Figure 3 shows how node-level and endpoint-level attributes map to protocol layers and their consuming functors.

3.1 Device Attributes (Non-Protocol)

These attributes describe the device itself, independent of any routing or switching protocol.

Constraints: hostname must match $[a-zA-Z][a-zA-Z0-9\-_]\{0,62\}$ (RFC 952/1123 hostname syntax). role values are advisory and do not affect functor behaviour; they are consumed by policy templates and reporting tools.

3.2 Interface/Endpoint Attributes (Non-Protocol)

These attributes describe interface identity and physical characteristics before any protocol configuration is applied.

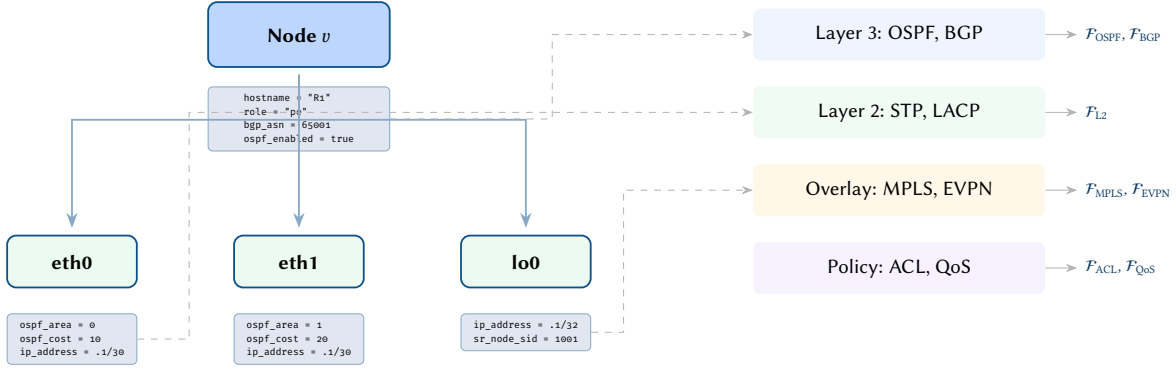


Figure 3. Attribute scope and layer mapping. Node-level attributes (top left) are stored in π_V ; endpoint-level attributes (bottom left) in π_E . Each protocol layer (right) reads from both scopes via its corresponding functor.

Table 2. Device (node-level) attributes.

Attribute	Type	Scope	R/O	Default	Description
hostname	string	node	R	—	Device hostname; must be unique within the topology
role	enum	node	O	—	Functional role: spine, leaf, border, core, access, pe, ce, rr
management_ip	ipv4_addr	node	O	—	Out-of-band management address
platform	string	node	O	—	Hardware platform identifier (e.g., “Nexus 9300”)
os_version	string	node	O	—	Operating system version string

Table 3. Interface/endpoint attributes.

Attribute	Type	Scope	R/O	Default	Description
name	string	endpoint	R	—	Interface name (e.g., “Ethernet1/1”, “Loopback0”)
description	string	endpoint	O	—	Administrative description
admin_status	enum	endpoint	O	up	up or down
mtu	u32	endpoint	O	1500	IP MTU in bytes
speed	enum	endpoint	O	—	10M, 100M, 1G, 10G, 25G, 40G, 100G, 400G
duplex	enum	endpoint	O	—	full, half, auto

Constraints: mtu range is 64–9216. name must be unique within the owning node. The pair (hostname, name) globally identifies an endpoint in G_{plan} .

3.3 IP Addressing (ip_*)

IP addressing attributes are endpoint-scoped and define the Layer 3 identity of each interface.

Table 4. IP addressing attributes.

Attribute	Type	Scope	R/O	Default	Description
ip_ipv4	ip_prefix	endpoint	O	—	Primary IPv4 address with prefix length
ip_ipv6	ip_prefix	endpoint	O	—	Primary IPv6 address with prefix length
ip_vrf	string	endpoint	O	“default”	VRF membership
ip_secondary_ipv4[n]	ip_prefix	endpoint	O	—	Secondary IPv4 addresses (indexed)
ip_ipv6_link_local	ipv6_addr	endpoint	O	auto	Link-local IPv6 (auto-derived from MAC if absent)

Constraints: At least one of ip_ipv4 or ip_ipv6 should be present on any routed interface. The prefix length must be valid for the address family (0–32 for IPv4, 0–128 for IPv6). Secondary addresses must share the same VRF as the primary. Point-to-point links conventionally use /31 (IPv4) or /127 (IPv6) prefixes.

3.4 OSPF (ospf_*)

OSPF attributes are split between node-level (process-wide settings) and endpoint-level (per-interface settings).

Constraints: ospf_dead must equal $4 \times \text{ospf_hello}$ on broadcast/NBMA networks (the validation pipeline enforces this as a warning). Both sides of a link must share the same ospf_area, ospf_hello, ospf_dead,

Table 5. OSPF node-level attributes.

Attribute	Type	Scope	R/O	Default	Description
ospf_enabled	bool	node	R	—	OSPF process active on this device
ospf_router_id	ipv4_addr	node	R	—	OSPF router ID (typically loopback address)
ospf_process_id	u32	node	O	1	OSPF process ID (local significance only)
ospf_reference_bandwidth	f64	node	O	100.0	Reference BW in Mbps for cost calculation
ospf_max_lsa	u32	node	O	—	Maximum LSAs before warning/shutdown
ospf_spf_delay	u32	node	O	200	Initial SPF delay in milliseconds
ospf_spf_hold	u32	node	O	5000	SPF hold time in milliseconds

Table 6. OSPF endpoint-level attributes.

Attribute	Type	Scope	R/O	Default	Description
ospf_area	u32	ep	R	—	OSPF area ID (dotted-decimal or integer)
ospf_area_type	enum	ep	O	normal	normal, stub, nssa, totally-stub, totally-nssa
ospf_cost	u32	ep	O	auto	Interface cost; auto = ref_bw / link speed
ospf_network_type	enum	ep	O	broadcast	broadcast, point-to-point, nbma, point-to-multipoint
ospf_hello	u32	ep	O	10	Hello interval in seconds
ospf_dead	u32	ep	O	40	Dead interval in seconds
ospf_priority	u32	ep	O	1	DR election priority (0 = never DR)
ospf_passive	bool	ep	O	false	Passive interface (suppress hello)
ospf_bfd	bool	ep	O	false	Enable BFD for OSPF on this interface
ospf_authentication	enum	ep	O	none	none, simple, md5, keychain
ospf_auth_key	string	ep	O	—	Authentication key or keychain name

ospf_network_type, and ospf_authentication settings; mismatches prevent adjacency formation and are flagged as errors. ospf_cost range is 1–65535. ospf_priority range is 0–255.

Functor interaction. $\mathcal{F}_{\text{OSPF}}$ reads the gate attribute ospf_enabled first; if absent or false, the entire OSPF subsection is skipped for that node. When enabled, the functor builds adjacency edges between endpoints that share the same ospf_area on opposite sides of an edge, inheriting cost and network type from the endpoint properties.

3.5 IS-IS (isis_*)

Table 7. IS-IS node-level attributes.

Attribute	Type	Scope	R/O	Default	Description
isis_enabled	bool	node	R	—	IS-IS routing enabled
isis_net	string	node	R	—	Network Entity Title (e.g., 49.0001.0010.0000.0001.00)
isis_level	enum	node	O	L1L2	L1, L2, L1L2
isis_overload_bit	bool	node	O	false	Set overload bit (ATT/OL)
isis_lsp_mtu	u32	node	O	1492	Maximum LSP size in bytes

Table 8. IS-IS endpoint-level attributes.

Attribute	Type	Scope	R/O	Default	Description
isis_metric	u32	ep	O	10	IS-IS metric for this interface
isis_circuit_type	enum	ep	O	L1L2	L1, L2, L1L2
isis_hello_interval	u32	ep	O	10	Hello interval in seconds
isis_hello_multiplier	u32	ep	O	3	Hello multiplier (hold = hello × multiplier)
isis_passive	bool	ep	O	false	Passive interface
isis_bfd	bool	ep	O	false	Enable BFD for IS-IS adjacency
isis_metric_style	enum	ep	O	wide	narrow, wide, transition

Constraints: `isis_net` must be a valid CLNS address with area ID, system ID (6 bytes), and NSEL=00. The `isis_circuit_type` on an endpoint must be compatible with the node-level `isis_level`: an L1-only router cannot have L2 circuits. `isis_metric` range is 1–16777215 for wide metrics, 1–63 for narrow metrics.

3.6 BGP (`bgp_*`)

BGP attributes are the most numerous in the schema, reflecting the protocol’s extensive configurability.

Table 9. BGP node-level attributes.

Attribute	Type	Scope	R/O	Default	Description
<code>bgp_enabled</code>	bool	node	R	—	BGP process active
<code>bgp_asn</code>	u32	node	R	—	Autonomous System Number
<code>bgp_router_id</code>	ipv4_addr	node	R	—	BGP router ID
<code>bgp_cluster_id</code>	ipv4_addr	node	O	—	Route reflector cluster ID
<code>bgp_confederation_id</code>	u32	node	O	—	Confederation identifier (public ASN)
<code>bgp_bestpath_as_path</code>	enum	node	O	—	ignore, multipath-relax
<code>bgp_bestpath_med</code>	enum	node	O	—	always-compare, deterministic
<code>bgp_graceful_restart</code>	bool	node	O	false	GR capability enabled
<code>bgp_gr_restart_time</code>	u32	node	O	120	GR restart time in seconds
<code>bgp_gr_stalepath_time</code>	u32	node	O	360	GR stale-path time in seconds

Table 10. BGP endpoint-level attributes.

Attribute	Type	Scope	R/O	Default	Description
<code>bgp_peer_addr</code>	ipv4_addr	ep	R	—	Neighbour IP address
<code>bgp_peer_asn</code>	u32	ep	R	—	Neighbour AS number
<code>bgp_peer_type</code>	enum	ep	O	auto	ibgp or ebgp; auto-derived from ASN comparison
<code>bgp_rr_client</code>	bool	ep	O	false	Treat this neighbour as an RR client
<code>bgp_rr_cluster_id</code>	ipv4_addr	ep	O	—	Per-peer cluster ID override
<code>bgp_multihop</code>	u32	ep	O	1	eBGP multihop TTL
<code>bgp_password</code>	string	ep	O	—	TCP MD5 authentication password
<code>bgp_update_source</code>	string	ep	O	—	Source interface for BGP session
<code>bgp_ipv4_unicast</code>	bool	ep	O	true	Activate IPv4 unicast AF
<code>bgp_ipv6_unicast</code>	bool	ep	O	false	Activate IPv6 unicast AF
<code>bgp_l2vpn_evpn</code>	bool	ep	O	false	Activate L2VPN EVPN AF
<code>bgp_send_community</code>	enum	ep	O	both	standard, extended, both, none
<code>bgp_route_map_in</code>	string	ep	O	—	Inbound route-map name
<code>bgp_route_map_out</code>	string	ep	O	—	Outbound route-map name
<code>bgp_prefix_list_in</code>	string	ep	O	—	Inbound prefix-list name
<code>bgp_prefix_list_out</code>	string	ep	O	—	Outbound prefix-list name
<code>bgp_default_originate</code>	bool	ep	O	false	Originate default route to peer
<code>bgp_soft_reconfiguration</code>	bool	ep	O	false	Enable soft reconfiguration inbound
<code>bgp_bfd</code>	bool	ep	O	false	Enable BFD for this BGP session
<code>bgp_timers_keepalive</code>	u32	ep	O	60	Keepalive interval in seconds
<code>bgp_timers_hold</code>	u32	ep	O	180	Hold time in seconds

Constraints: `bgp_asn` range is 1–4294967295 (4-byte ASN). `bgp_timers_hold` must be either 0 (infinite) or at least $3 \times \text{bgp_timers_keepalive}$. Both ends of a BGP session must agree on `bgp_password` (present or absent on both sides). If `bgp_rr_client` is true, the local node must have `bgp_cluster_id` set. `bgp_peer_type` is auto-derived: ibgp when local ASN equals peer ASN, ebgp otherwise; explicit setting overrides this (e.g., for confederations).

Functor interaction. $\mathcal{F}_{\text{BGP}}$ builds a session graph by pairing endpoints whose `bgp_peer_addr` values match the remote node’s loopback or interface IP. iBGP sessions inherit the IGP next-hop for route resolution; eBGP sessions use the directly connected address. The functor marks route-reflector hierarchies using `bgp_rr_client` and `bgp_cluster_id`.

3.7 MPLS and Segment Routing (`mpls_*`, `sr_*`)

Constraints: `sr_global_block_start` must be less than `sr_global_block_end`. The SRGB range must not overlap with the dynamic label range. `sr_node_sid` must be unique across all nodes in the IGP do-

Table 11. MPLS and SR node-level attributes.

Attribute	Type	Scope	R/O	Default	Description
mpls_enabled	bool	node	R	—	MPLS forwarding enabled globally
mpls_label_range_start	u32	node	O	16	Start of dynamic label range
mpls_label_range_end	u32	node	O	1048575	End of dynamic label range
mpls_ldp_enabled	bool	node	O	false	LDP enabled globally
mpls_rsvp_enabled	bool	node	O	false	RSVP-TE enabled globally
sr_enabled	bool	node	O	false	Segment Routing enabled
sr_global_block_start	u32	node	O	16000	SRGB start label
sr_global_block_end	u32	node	O	23999	SRGB end label
sr_node_sid	u32	node	O	—	Node segment identifier (prefix SID index)

Table 12. MPLS and SR endpoint-level attributes.

Attribute	Type	Scope	R/O	Default	Description
mpls_ldp_enabled	bool	ep	O	false	LDP enabled on this interface
sr_adj_sid	u32	ep	O	—	Adjacency SID (static or auto)
mpls_te_metric	u32	ep	O	—	TE metric (distinct from IGP metric)
mpls_te_admin_group	string	ep	O	—	Admin group bitmask (e.g., “0x0000000F”)
rsvp_te_bandwidth	f64	ep	O	—	Reservable bandwidth in Mbps

main. mpls_label_range_start minimum is 16 (labels 0–15 are reserved). If sr_enabled is true, either ospf_enabled or isis_enabled must also be true (SR requires an IGP).

Functor interaction. $\mathcal{F}_{\text{MPLS}}$ reads sr_enabled and mpls_enabled to determine whether to produce an SR-MPLS or LDP-based overlay. Node SIDs are allocated from the SRGB range (sr_global_block_start to sr_global_block_end); the functor verifies uniqueness and range containment across all nodes.

3.8 EVPN (evpn_*)

Table 13. EVPN node-level attributes.

Attribute	Type	Scope	R/O	Default	Description
evpn_enabled	bool	node	R	—	EVPN control plane active
evpn_anycast_mac	mac_addr	node	O	—	Anycast gateway MAC (shared across VTEPs)
evpn_irb_mode	enum	node	O	symmetric	symmetric or asymmetric IRB

Table 14. EVPN endpoint-level attributes.

Attribute	Type	Scope	R/O	Default	Description
evpn_vni	u32	ep	R	—	VXLAN Network Identifier
evpn_rd	string	ep	O	auto	Route distinguisher (type:value)
evpn_rt_import	string	ep	O	auto	Import route target
evpn_rt_export	string	ep	O	auto	Export route target
evpn_esi	string	ep	O	—	Ethernet Segment Identifier (10-byte hex)
evpn_esi_mode	enum	ep	O	type-1	type-1 (manual) or type-3 (auto)
evpn_l3vni	u32	ep	O	—	L3 VNI for inter-subnet routing
evpn_gw_role	enum	ep	O	distributed	centralised, distributed, none
evpn_anycast_ip	ipv4_addr	ep	O	—	Anycast gateway IP for this VLAN
evpn_arp_suppress	bool	ep	O	true	ARP suppression enabled

Constraints: evpn_vni range is 1–16777215. evpn_esi must be a 10-byte value in colon-separated hex (e.g., 00:11:22:33:44:55:66:77:88:99). If evpn_enabled is true, then bgp_enabled must also be true and bgp_l2vpn_evpn must be activated on at least one BGP session. Symmetric IRB requires evpn_l3vni on every participating VTEP. evpn_anycast_mac should be identical across all nodes sharing anycast gateway responsibilities.

3.9 STP (stp_*)

Constraints: stp_priority must be a multiple of 4096, range 0–61440. stp_port_priority must be a multiple of 16, range 0–240. stp_portfast should only be enabled on access-facing ports (not uplinks); the

Table 15. STP node-level attributes.

Attribute	Type	Scope	R/O	Default	Description
stp_enabled	bool	node	R	—	Spanning tree active
stp_mode	enum	node	O	rstp	stp, rstp, mstp, pvst, rpvt
stp_priority	u32	node	O	32768	Bridge priority (in multiples of 4096)

Table 16. STP endpoint-level attributes.

Attribute	Type	Scope	R/O	Default	Description
stp_port_priority	u32	ep	O	128	Port priority (multiples of 16)
stp_port_cost	u32	ep	O	auto	Path cost (auto from speed)
stp_guard	enum	ep	O	none	root, loop, bpdu, none
stp_portfast	bool	ep	O	false	PortFast (immediate forwarding)
stp_bpdu_filter	bool	ep	O	false	Suppress BPDU transmission
stp_bpdu_guard	bool	ep	O	false	Err-disable on BPDU receipt

validation pipeline warns if `stp_portfast` is set on a trunk port. `stp_bpdu_guard` and `stp_guard=bpdu` are equivalent representations; do not set both.

3.10 L2 / Switching (`switchport_*`, `vlan_*`, `lACP_*`)

Table 17. Layer 2 switching endpoint-level attributes.

Attribute	Type	Scope	R/O	Default	Description
switchport	enum	ep	O	—	access, trunk, hybrid
access_vlan	u32	ep	O	1	Access VLAN ID
allowed_vlans	string	ep	O	"all"	Comma-separated VLAN list or "all"
native_vlan	u32	ep	O	1	Native (untagged) VLAN on trunks
lACP_mode	enum	ep	O	—	active, passive, on
lACP_rate	enum	ep	O	normal	fast (1s) or normal (30s)
lACP_system_priority	u32	ep	O	32768	LACP system priority
lACP_port_priority	u32	ep	O	32768	LACP port priority
lldp_enabled	bool	ep	O	true	LLDP transmit and receive

Constraints: `access_vlan` range is 1–4094. `allowed_vlans` accepts comma-separated integers and ranges (e.g., "100,200-210,300"). If `switchport=access`, `allowed_vlans` is ignored. `lACP_system_priority` range is 1–65535. Both members of a LAG must agree on `lACP_mode` (both active, or one active and one passive).

3.11 VRRP/HSRP (`vrrp_*`, `hsrp_*`)

Table 18. FHRP (VRRP/HSRP) endpoint-level attributes.

Attribute	Type	Scope	R/O	Default	Description
vrrp_enabled	bool	ep	R	—	VRRP active on this interface
vrrp_group	u32	ep	R	—	VRRP group ID
vrrp_virtual_ip	ipv4_addr	ep	R	—	Virtual IP address
vrrp_priority	u32	ep	O	100	VRRP priority (1–254)
vrrp_preempt	bool	ep	O	true	Preemption enabled
vrrp_version	enum	ep	O	3	2 or 3
vrrp_timers_advertise	u32	ep	O	1	Advertisement interval in seconds
hsrp_enabled	bool	ep	R	—	HSRP active on this interface
hsrp_group	u32	ep	R	—	HSRP group ID
hsrp_virtual_ip	ipv4_addr	ep	R	—	Virtual IP address
hsrp_priority	u32	ep	O	100	HSRP priority (0–255)
hsrp_preempt	bool	ep	O	false	Preemption enabled

Constraints: `vrrp_group` range is 1–255 (VRRPv2) or 1–4094 (VRRPv3). `hsrp_group` range is 0–255 (v1) or 0–4095 (v2). `vrrp_virtual_ip` must be in the same subnet as `ip_ipv4` on the endpoint. All endpoints

sharing a VRRP/HSRP group on the same broadcast domain must use the same virtual IP address. Only one of VRRP or HSRP should be active per interface.

3.12 QoS (qos_*)

Table 19. QoS attributes.

Attribute	Type	Scope	R/O	Default	Description
qos_enabled	bool	node	R	—	QoS processing active
qos_trust	enum	ep	O	dscp	cos, dscp, ip-precedence
qos_default_cos	u32	ep	O	0	Default CoS marking (0–7)
qos_default_dscp	u32	ep	O	0	Default DSCP marking (0–63)
qos_ingress_policy	string	ep	O	—	Ingress policy-map name
qos_egress_policy	string	ep	O	—	Egress policy-map name
qos_shape_rate	u64	ep	O	—	Shaper rate in bps
qos_police_rate	u64	ep	O	—	Policer rate in bps
qos_police_burst	u32	ep	O	—	Policer burst size in bytes

Constraints: qos_default_cos range is 0–7. qos_default_dscp range is 0–63. qos_shape_rate and qos_police_rate must not exceed the interface speed. Policy-map names referenced by qos_ingress_policy and qos_egress_policy must correspond to defined policy objects (this is validated as a cross-reference check).

3.13 ACL (acl_*)

ACL definitions are node-scoped (they define the ruleset), while ACL application is endpoint-scoped (binding a named ACL to an interface direction).

Table 20. ACL node-level attributes (per-rule, indexed by name and sequence).

Attribute Pattern	Type	Scope	R/O	Description
acl_[name]_seq_[n]_action	enum	node	R	permit or deny
acl_[name]_seq_[n]_protocol	enum	node	R	ip, tcp, udp, icmp, etc.
acl_[name]_seq_[n]_src	string	node	R	Source: IP prefix or any
acl_[name]_seq_[n]_dst	string	node	R	Destination: IP prefix or any
acl_[name]_seq_[n]_src_port	string	node	O	Source port or range (e.g., “eq 80”, “range 1024 65535”)
acl_[name]_seq_[n]_dst_port	string	node	O	Destination port or range

Table 21. ACL endpoint-level attributes (application).

Attribute	Type	Scope	R/O	Default	Description
acl_ingress	string	ep	O	—	Name of ACL applied inbound
acl_egress	string	ep	O	—	Name of ACL applied outbound

Constraints: Sequence numbers [n] must be positive integers, typically multiples of 10. ACL names referenced by acl_ingress and acl_egress must have at least one rule defined in the node-level ACL attributes. Each ACL should end with an explicit deny ip any any rule (the validation pipeline warns if this implicit deny is not made explicit).

3.14 Security (sgt_*, macsec_*, dot1x_*)

Table 22. SGT (Scalable Group Tag) attributes.

Attribute	Type	Scope	R/O	Default	Description
sgt_enabled	bool	node	R	—	SGT enforcement active
sgt_value	u32	ep	O	—	Static SGT assignment (1–65535)
sgt_propagation	enum	ep	O	inline	inline (CMD tag) or sxp
sgt_enforcement	bool	ep	O	false	SGACL enforcement on this port

Constraints: sgt_value range is 1–65535 (0 is “unknown”). Both sides of a MACsec-protected link must agree on macsec_cipher and macsec_key_name. dot1x_guest_vlan and dot1x_auth_fail_vlan must be valid VLAN IDs (1–4094) if set. 802.1X should only be configured on access-facing ports, not on inter-switch links.

Table 23. MACsec attributes.

Attribute	Type	Scope	R/O	Default	Description
macsec_enabled	bool	ep	R	—	MACsec active on this link
macsec_policy	enum	ep	O	should-secure	must-secure, should-secure
macsec_cipher	enum	ep	O	gcm-aes-128	gcm-aes-128, gcm-aes-256, gcm-aes-xpn-128, gcm-aes-xpn-256
macsec_key_name	string	ep	O	—	Pre-shared key name or MKA keychain
macsec_replay_window	u32	ep	O	0	Anti-replay window size

Table 24. 802.1X attributes.

Attribute	Type	Scope	R/O	Default	Description
dot1x_enabled	bool	ep	R	—	802.1X authentication active
dot1x_mode	enum	ep	O	single-host	single-host, multi-host, multi-auth, multi-domain
dot1x_reauthentication	bool	ep	O	true	Periodic re-authentication
dot1x_reauth_period	u32	ep	O	3600	Re-auth interval in seconds
dot1x_guest_vlan	u32	ep	O	—	VLAN for unauthenticated hosts
dot1x_auth_fail_vlan	u32	ep	O	—	VLAN for authentication failures
dot1x_mab	bool	ep	O	false	MAC Authentication Bypass fallback
dot1x_order	enum	ep	O	dot1x-mab	Authentication order: dot1x-mab or mab-dot1x

Functor interaction. $\mathcal{F}_{\text{Security}}$ reads SGT, MACsec, and 802.1X attributes to build a policy overlay. SGT assignments flow through the graph to identify trust boundaries; MACsec settings must be symmetric across each link. The 802.1X attributes inform the L2 overlay by determining whether a port requires authentication before forwarding.

3.15 Physical Layer (phy_*)

Table 25. Physical layer endpoint-level attributes.

Attribute	Type	Scope	R/O	Default	Description
phy_medium	enum	ep	O	—	copper, fiber, virtual
phy_connector	enum	ep	O	—	rj45, sfp, sfp+, sfp28, qsfp+, qsfp28, qsfp-dd, cfp, osfp
phy_fec	enum	ep	O	none	none, rs (Reed-Solomon), fc (fire code)
phy_auto_negotiation	bool	ep	O	true	IEEE 802.3 auto-negotiation
dom_rx_power_threshold	f64	ep	O	—	DOM receive power alarm threshold in dBm
dom_tx_power_threshold	f64	ep	O	—	DOM transmit power alarm threshold in dBm

Constraints: phy_fec is only relevant for 25G+ links; the validation pipeline warns if FEC is set on 1G or 10G copper interfaces. Both sides of a link must agree on phy_auto_negotiation (both enabled or both disabled); a mismatch is a common cause of link failures and is flagged as an error. DOM thresholds are typically negative values (e.g., −8.0 dBm).

4 Design Rules: Primitives and Assertions

The attribute catalogue (§3) defines the raw mathematical variables of G_{plan} . However, network architects do not think in flat lists of attributes; they think in *Design Rules*. Design Rules form the crucial translation layer between topological specification and the formal attribute schema.

We formally categorise Design Rules into two components:

- **Primitives (The Nouns):** A Primitive is a cohesive bundle of attributes that represents a single, self-contained network feature (e.g., an EVPN_Symmetric_IRB Primitive, or a BGP_Route_Reflector Primitive). Instantiating a Primitive guarantees that all mandatory attributes for that feature are populated in the graph. The Design Pattern Catalogue (§5) serves as the canonical library of these Primitives.
- **Assertions (The Verbs/Constraints):** An Assertion is a strict mathematical guarantee applied to the graph. Assertions govern how Primitives are allowed to interact. For example, a structural Assertion might enforce that “A MACsec Primitive can only be applied to an endpoint if its peer endpoint also

has a matching MACsec Primitive.” The Validation Rules (§6) are the computational implementation of these Assertions.

By restricting graph mutations to these standardised Primitives and Assertions, the compilation pipeline guarantees that the resulting plan topology is correct-by-construction and perfectly aligned with the architect’s original structural intent.

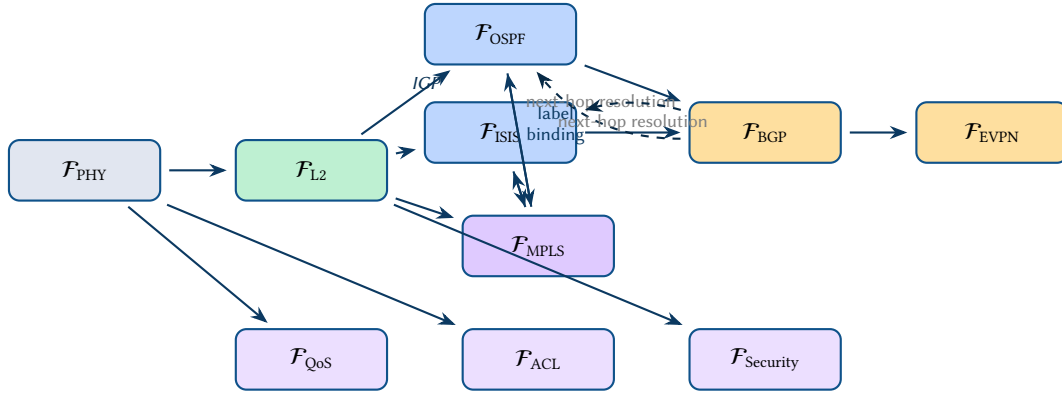
5 Design Pattern Catalogue

This section presents reusable topology patterns that serve as building blocks for real network designs. Each pattern specifies a plan topology fragment with concrete attribute values. Patterns can be composed: the composite patterns in §5.7 show how multiple single-protocol patterns overlay on the same G_{plan} .

For each pattern we provide:

- A brief description and when to use it.
- A TikZ diagram of the plan topology fragment.
- A table of key attribute values.
- Notes on functor output and common variants.

The patterns below are organised by protocol family in the same dependency order as the functor DAG (Figure 4): BGP patterns first (since BGP is often the inter-domain glue), then IGP, MPLS/SR, EVPN-VXLAN, campus/access, and finally composite patterns that combine multiple protocol layers.



Topological ordering determines functor evaluation sequence.
Cross-layer functors (bottom row) read from G_{plan} directly.

Figure 4. Functor dependency DAG: directed edges show ‘depends-on’ relationships between layer functors. The IGP layer offers two alternative functors (F_{OSPF} and F_{ISIS}) which feed into F_{BGP} and F_{MPLS} . Cross-layer functors (F_{QoS} , F_{ACL} , F_{Security}) read from the plan topology and physical/L2 layers.

5.1 BGP Patterns

5.1.1 iBGP Full Mesh

Description. Every router peers with every other router via iBGP. All routers share the same ASN. No route reflectors are needed.

When to use. Small networks with 2–5 routers where the $O(n^2)$ session count is manageable.

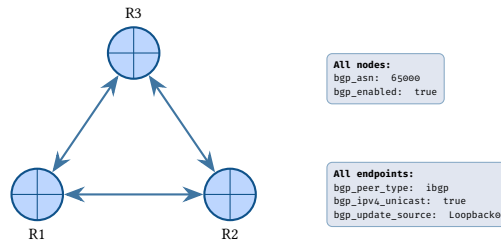


Figure 5. iBGP full mesh among three routers.

Functor output. F_{BGP} produces a complete graph on all three nodes with iBGP edges. The next-hop-self behaviour is implicit for iBGP; the functor annotates each overlay edge with the session type.

Variants. For IPv6, add `bgp_ipv6_unicast=true` on each endpoint. For dual-stack, activate both address families.

Table 26. Key attributes for iBGP full mesh.

Attribute	Value	Notes
bgp_asn	65000 (same on all)	Single AS
bgp_peer_type	ibgp	Auto-derived since ASNs match
bgp_update_source	Loopback0	Peer via loopbacks for stability
bgp_ipv4_unicast	true	IPv4 unicast AF active
bgp_send_community	both	Full community propagation

5.1.2 Single Route Reflector

Description. One router acts as a route reflector (RR). All other routers peer only with the RR, reducing the session count from $O(n^2)$ to $O(n)$.

When to use. Medium networks (5–50 routers) where a full mesh is impractical but a single RR provides sufficient redundancy (paired with a backup RR in production).

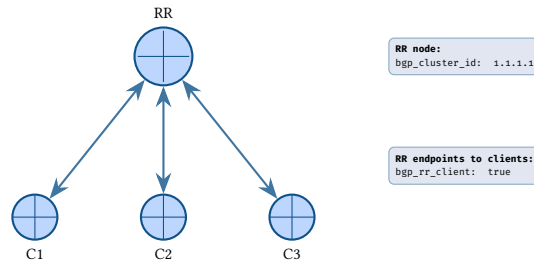


Figure 6. Single route reflector with three clients.

Table 27. Key attributes for single route reflector.

Attribute	On RR	On Clients	Notes
bgp_asn	65000	65000	Same AS
bgp_cluster_id	1.1.1.1	—	Only on RR
bgp_rr_client	true (on client-facing eps)	—	RR marks its client peers
bgp_update_source	Loopback0	Loopback0	Peer via loopbacks

Functor output. $\mathcal{F}_{\text{BGP}}$ produces a star graph with the RR at the centre. Overlay edges from RR to clients are annotated with `rr_client=true`, enabling the BGP best-path decision process to apply route-reflection rules (no AS-path loop check for reflected routes).

Variants. Dual RR: deploy two RRs in the same cluster (same `bgp_cluster_id`) for redundancy. Each client peers with both RRs.

5.1.3 Hierarchical Route Reflectors

Description. A two-tier RR hierarchy where top-level RRs peer with each other in a full mesh, and each top-level RR serves a set of lower-level RRs, which in turn serve leaf clients.

When to use. Large service-provider networks (100+ routers) with regional aggregation points.

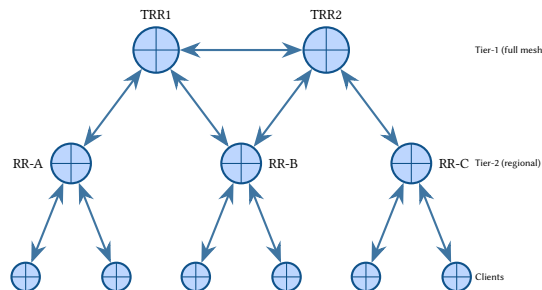


Figure 7. Hierarchical route reflectors: two tiers.

Each tier-1 RR has `bgp_cluster_id` set to a unique value and marks tier-2 RRs as `bgp_rr_client=true`. Each tier-2 RR has its own `bgp_cluster_id` and marks leaf clients as `bgp_rr_client=true`. The distinct cluster IDs at each tier prevent routing loops in the reflection path.

Functor output. A three-level tree overlay. The functor validates that cluster IDs are distinct across tiers and that the reflection graph is loop-free.

5.1.4 eBGP Underlay (Clos/Leaf-Spine)

Description. Each device has a unique ASN. eBGP sessions run on every physical link. This is the standard data-centre underlay design.

When to use. Data-centre Clos fabrics where simplicity and fast convergence are priorities.

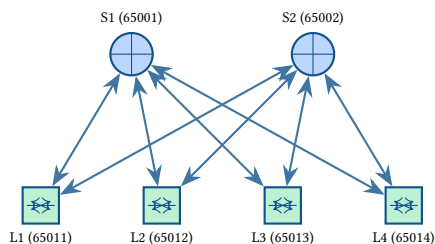


Figure 8. eBGP underlay: unique ASN per device in a leaf-spine fabric.

Table 28. Key attributes for eBGP Clos underlay.

Attribute	Value	Notes
bgp_asn	Unique per device	e.g., 65001 for S1, 65011 for L1
bgp_peer_type	ebgp	All sessions are eBGP
bgp_peer_addr	Interface IP	Direct peering, no loopback
bgp_bestpath_as_path	multipath-relax	Enable ECMP across ASNs
bgp_multihop	1	Single-hop eBGP

Functor output. A bipartite graph (spines ↔ leaves) with eBGP edges. With multipath-relax, ECMP is available across all spine paths.

Variants. Some designs use a shared spine ASN (e.g., 65000 for all spines) with allowas-in on leaves. The schema supports this by setting the same bgp_asn on all spine nodes.

5.1.5 BGP Confederations

Description. The AS is divided into sub-ASes. Within each sub-AS, iBGP or RR is used. Between sub-ASes, confederation-eBGP sessions run.

When to use. Large enterprises that want to reduce iBGP session count without route reflectors, or ISPs merging multiple ASes.

In a confederation, each sub-AS operates an independent iBGP mesh (or route reflector hierarchy). Confederation-eBGP sessions between sub-ASes behave like eBGP for path selection but preserve iBGP-like next-hop and local-preference semantics. The functor distinguishes these sessions by checking whether both endpoints share the same bgp_confederation_id even though their bgp_asn values differ. Figure 9 illustrates the topology.

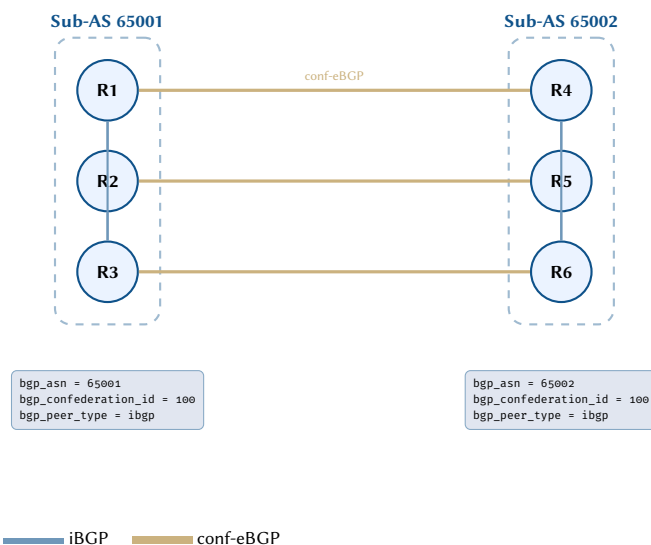


Figure 9. BGP confederation topology. Routers within each sub-AS form an iBGP mesh; routers across sub-AS boundaries establish confederation-eBGP sessions. All members share the same bgp_confederation_id.

Table 29. Key attributes for BGP confederations.

Attribute	Value	Notes
bgp_asn	Sub-AS number (e.g., 65001)	Per-device sub-AS
bgp_confederation_id	100 (public AS)	Same on all members
bgp_peer_type	ibgp or ebgp	ibgp within sub-AS, ebgp between

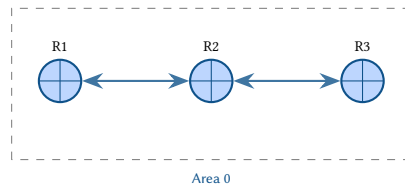
The functor recognises confederation-eBGP sessions when the `bgp_confederation_id` is set and the peer ASN differs from the local ASN but both are within the confederation. These sessions preserve the next-hop and local-preference attributes, unlike regular eBGP.

5.2 IGP Patterns

5.2.1 OSPF Single Area

Description. All interfaces belong to OSPF Area 0. The simplest OSPF deployment.

When to use. Small to medium networks (up to 50 routers) where the LSDB size is manageable.

**Figure 10.** OSPF single area (Area 0).**Table 30.** Key attributes for OSPF single area.

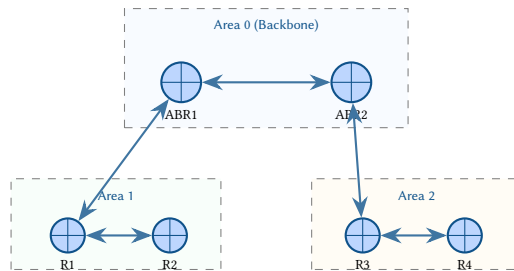
Attribute	Value	Notes
ospf_enabled	true	On all nodes
ospf_router_id	Loopback IP	Unique per router
ospf_area	0	All endpoints in backbone
ospf_network_type	point-to-point	For /31 links; broadcast for multi-access
ospf_passive	true	On host-facing interfaces

Functor output. $\mathcal{F}_{\text{OSPF}}$ produces a single-area overlay graph. Every OSPF-enabled endpoint with a matching `ospf_area` on the far side forms an adjacency edge.

5.2.2 OSPF Multi-Area (Hub-and-Spoke)

Description. Area 0 serves as the backbone. Each remote site is placed in its own non-backbone area. ABRs connect non-backbone areas to Area 0.

When to use. Enterprise networks with a central campus and branch offices, or any topology requiring LSDB partitioning.

**Figure 11.** OSPF multi-area hub-and-spoke.

ABR nodes have endpoints in multiple areas. For example, ABR1's uplink to ABR2 has `ospf_area=0`, while its downlink to R1 has `ospf_area=1`. The functor produces inter-area adjacency edges and marks ABR nodes in the overlay.

5.2.3 OSPF Stub/NSSA Areas

Description. Non-backbone areas configured as stub or NSSA to reduce LSDB size by filtering external LSAs.

When to use. Branch offices with a single exit point (stub) or branches that inject external routes via redistribution (NSSA).

Stub areas block Type-5 (AS-external) LSAs at the ABR, replacing them with a default route. NSSA areas similarly block Type-5 LSAs but allow internal routers to inject external routes as Type-7 LSAs, which the ABR converts to Type-5 for the backbone. The functor checks that all endpoints within an area agree on `ospf_area_type` and applies the appropriate LSA filtering rules in the overlay. Figure 12 shows the topology with blocked and permitted LSA flows.

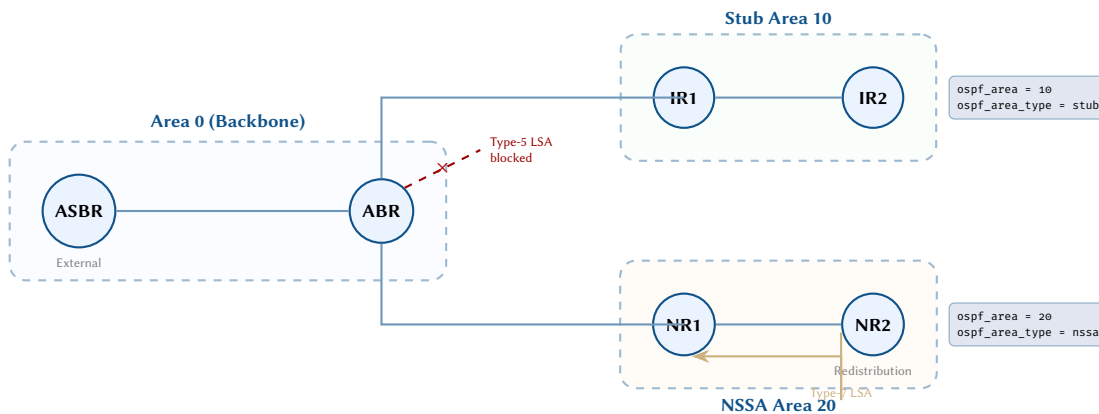


Figure 12. OSPF stub and NSSA area topology. The ABR straddles Area 0 and the non-backbone areas. Type-5 LSAs are blocked at the stub area boundary (red dashed); Type-7 LSAs carry external routes within the NSSA area.

Table 31. Key attributes for OSPF stub/NSSA areas.

Attribute	Value	Notes
<code>ospf_area</code>	Non-zero	e.g., 10
<code>ospf_area_type</code>	<code>stub</code>	No Type-5 LSAs; default route injected by ABR
<code>ospf_area_type</code>	<code>nssa</code>	No Type-5; Type-7 for external routes
<code>ospf_area_type</code>	<code>totally-stub</code>	No Type-3/5; only default from ABR
<code>ospf_area_type</code>	<code>totally-nssa</code>	No Type-3/5; Type-7 for locals

All endpoints within the area must agree on `ospf_area_type`. The validation pipeline checks this and raises an error on mismatch.

5.2.4 IS-IS Multi-Level

Description. IS-IS Level 1 is used within areas, Level 2 forms the backbone. L1/L2 routers act as area boundaries.

When to use. Service-provider cores and large-scale fabrics where IS-IS is preferred over OSPF for its flat TLV extensibility and faster convergence.

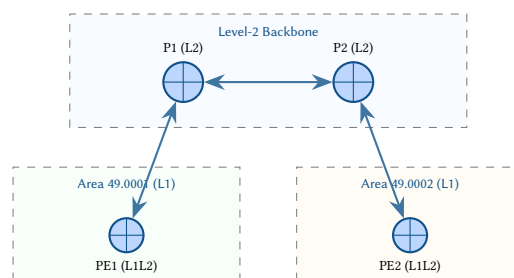


Figure 13. IS-IS multi-level with L1/L2 boundary routers.

Table 32. Key attributes for IS-IS multi-level.

Attribute	Value	Notes
<code>isis_enabled</code>	<code>true</code>	All nodes
<code>isis_net</code>	49.0001...	Area-specific NET
<code>isis_level</code>	L2 (core), L1L2 (boundary)	Node-level
<code>isis_circuit_type</code>	L2 (core links), L1 (area links)	Per-interface
<code>isis_metric_style</code>	<code>wide</code>	Wide metrics for SR compatibility

5.2.5 IGP-BGP Redistribution

Description. Routes are redistributed between an IGP (OSPF or IS-IS) and BGP at PE routers. Commonly used at the border between the enterprise IGP domain and the WAN provider BGP domain.

When to use. Enterprise-to-provider hand-off, or legacy networks migrating from IGP to BGP.

The redistribution is modelled via route-map attributes on the BGP endpoints:

Table 33. Key attributes for IGP-BGP redistribution at a PE.

Attribute	Value	Notes
ospf_enabled	true	IGP side
bgp_enabled	true	BGP side
bgp_route_map_out	RM-OSPF-TO-BGP	Filters redistributed routes
bgp_route_map_in	RM-BGP-TO-OSPF	Filters incoming BGP routes

The functor does not perform redistribution itself; it records the route-map references so that the redistribution policy can be validated and compiled by downstream tooling.

5.3 MPLS/SR Patterns

5.3.1 LDP-over-OSPF

Description. MPLS labels are distributed automatically by LDP, which follows the IGP (OSPF) topology. Every OSPF interface also runs LDP.

When to use. Traditional MPLS service-provider cores before the adoption of Segment Routing.

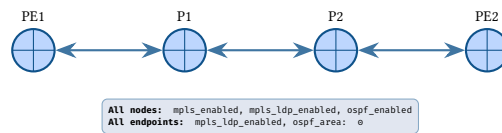


Figure 14. LDP-over-OSPF MPLS core.

Table 34. Key attributes for LDP-over-OSPF.

Attribute	Value	Notes
mpls_enabled	true	All nodes
mpls_ldp_enabled	true	Node and all core-facing endpoints
ospf_enabled	true	IGP for label distribution
ospf_area	0	Single-area backbone

Functor output. $\mathcal{F}_{\text{MPLS}}$ produces an MPLS overlay with LDP-signalled LSPs. Each edge in the overlay carries a label binding derived from the IGP prefix.

5.3.2 SR-MPLS with TI-LFA

Description. Segment Routing replaces LDP. Each node advertises a node SID via the IGP (OSPF or IS-IS). Topology-Independent Loop-Free Alternate (TI-LFA) provides sub-50ms failover.

When to use. Modern MPLS cores and data-centre fabrics requiring simplified label management and fast convergence.

Each node advertises a globally unique node SID via the IGP, eliminating the per-hop label distribution protocol (LDP). TI-LFA pre-computes a backup path for every prefix using the post-convergence SPT, ensuring sub-50 ms failover without the tunnel overhead of RSVP-TE FRR. The functor reads `sr_node_sid` and `sr_tilfa_enabled` to construct the primary and backup forwarding entries. Figure 15 illustrates the primary path and the TI-LFA backup via an alternate node.

Functor output. The MPLS overlay includes node SID labels and adjacency SID labels. TI-LFA backup paths are computed by the functor from the IGP topology.

5.3.3 RSVP-TE with FRR

Description. RSVP-TE signals explicit-path LSPs with bandwidth reservations. Fast Reroute (FRR) provides facility-backup or one-to-one backup tunnels for link/node protection.

When to use. Traffic-engineered cores requiring bandwidth guarantees and deterministic paths.

RSVP-TE differs from LDP in that LSPs are signalled with explicit path constraints (bandwidth, admin groups, hop limits). FRR pre-computes backup tunnels at each PLR (Point of Local Repair) to protect against link or

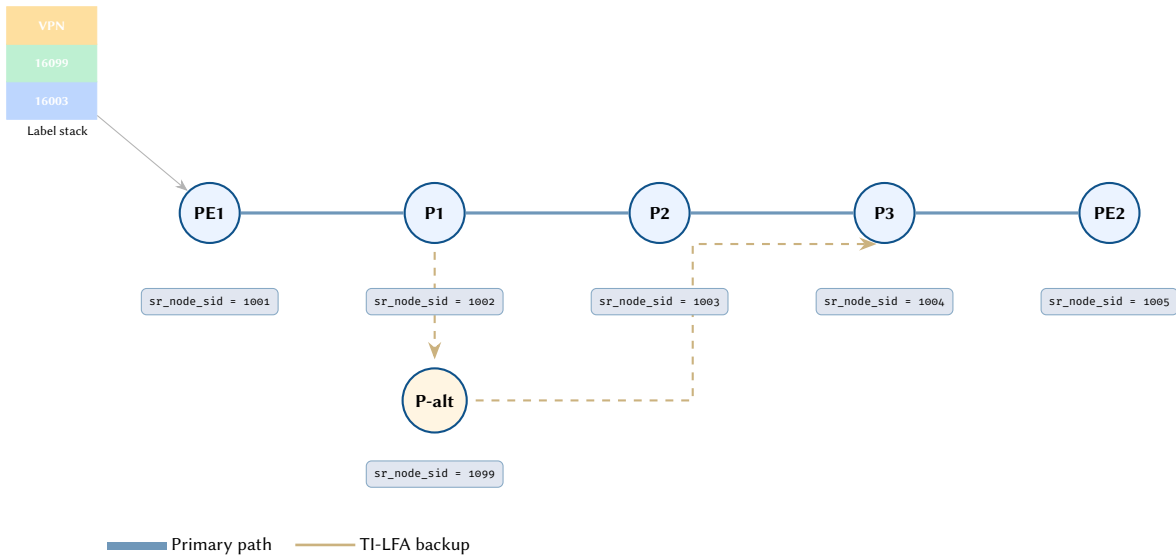


Figure 15. SR-MPLS with TI-LFA. The primary path runs PE1–P1–P2–P3–PE2. If the P1–P2 link fails, TI-LFA routes traffic via the pre-computed backup through P-alt. Each node’s sr_node_sid identifies its segment; the label stack (left) shows the MPLS encapsulation at PE1.

Table 35. Key attributes for SR-MPLS with TI-LFA.

Attribute	Value	Notes
sr_enabled	true	All nodes
sr_global_block_start	16000	SRGB start
sr_global_block_end	23999	SRGB end
sr_node_sid	Unique per node	e.g., 1001, 1002, ...
sr_adj_sid	Per-endpoint	Auto or static adjacency SIDs
isis_enabled	true	IS-IS as the IGP (most common for SR)
isis_metric_style	wide	Required for SR TLV advertisement

node failures. The functor reads rsvp_te_bandwidth and mpls_te_admin_group to construct constrained shortest paths and verify that backup tunnels exist for protected segments.

Table 36. Key attributes for RSVP-TE with FRR.

Attribute	Value	Notes
mpls_enabled	true	All nodes
mpls_rsvp_enabled	true	All nodes
rsvp_te_bandwidth	e.g., 1000.0	Reservable BW per interface (Mbps)
mpls_te_metric	e.g., 10	TE metric (may differ from IGP metric)
mpls_te_admin_group	e.g., “0x01”	Affinity/admin group for CSPF

5.4 EVPN-VXLAN Patterns

5.4.1 Symmetric IRB with Distributed Gateway

Description. Every VTEP performs both bridging and routing. An L3 VNI carries inter-subnet traffic symmetrically between ingress and egress VTEPs. All VTEPs share an anycast gateway MAC and IP.

When to use. The standard EVPN-VXLAN data-centre design for multi-tenant fabrics.

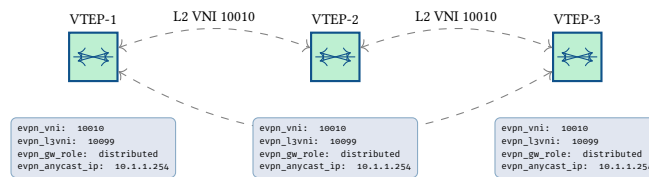


Figure 16. Symmetric IRB with distributed anycast gateway.

Table 37. Key attributes for symmetric IRB.

Attribute	Value	Notes
evpn_enabled	true	All VTEPs
evpn_irb_mode	symmetric	Node-level
evpn_vni	10010	L2 VNI per VLAN
evpn_l3vni	10099	L3 VNI for inter-subnet routing
evpn_gw_role	distributed	Every VTEP routes
evpn_anycast_mac	00:00:5e:00:01:01	Shared across all VTEPs
evpn_anycast_ip	10.1.1.254	Shared gateway IP per VLAN
evpn_arp_suppress	true	Suppress ARP flooding

5.4.2 Asymmetric IRB with Centralised Gateway

Description. Only designated gateway VTEPs perform routing. Non-gateway VTEPs bridge to the gateway for inter-subnet traffic. No L3 VNI is needed since routing occurs only at the gateway.

When to use. Simpler deployments where a pair of centralised gateways is acceptable, or hardware that does not support symmetric IRB.

In asymmetric IRB, inter-subnet traffic from a leaf VTEP is bridged to the centralised gateway, which performs routing and bridges the return traffic back. This “hairpin” path adds latency but simplifies the leaf configuration: no L3 VNI or distributed anycast gateway is needed. The functor distinguishes gateway VTEPs (`evpn_gw_role = centralised`) from bridge-only leaves (`evpn_gw_role = none`) and models the asymmetric forwarding path accordingly. Figure 17 shows the topology.

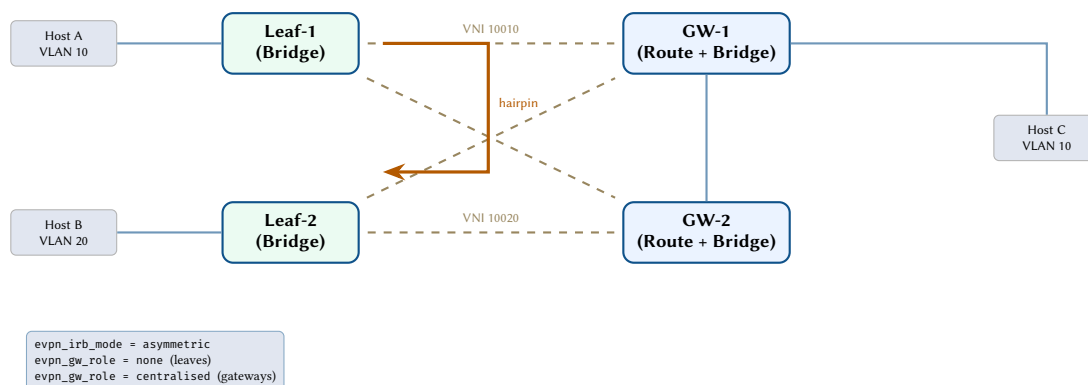


Figure 17. EVPN asymmetric IRB with centralised gateways. Leaf VTEPs perform bridging only; inter-subnet traffic hairpins through a gateway VTEP that performs routing. VXLAN tunnels (dashed) connect all VTEPs.

Table 38. Key attributes for asymmetric IRB with centralised gateway.

Attribute	Gateway VTEPs	Leaf VTEPs	Notes
evpn_irb_mode	asymmetric	asymmetric	Node-level
evpn_gw_role	centralised	none	Only gateways route
evpn_l3vni	—	—	Not required
evpn_vni	10010	10010	Same L2 VNI

Functor output. The EVPN overlay marks gateway nodes with the centralised role. Inter-subnet routing is modelled as a path through the gateway, not as a direct VTEP-to-VTEP path.

5.4.3 EVPN Multi-Homing (ESI LAG)

Description. A host or server connects to two VTEPs via a LAG. Both VTEPs advertise the same Ethernet Segment Identifier (ESI) so that the fabric can load-balance and provide redundancy.

When to use. Server dual-homing in EVPN-VXLAN fabrics for active-active redundancy.

5.5 Campus/Access Patterns

5.5.1 STP Root Placement (Primary/Secondary)

Description. Two distribution switches are configured with low bridge priorities to become the primary and secondary STP root bridges. All other switches retain the default priority.

When to use. Any campus network with Layer 2 loops that requires deterministic STP topology.

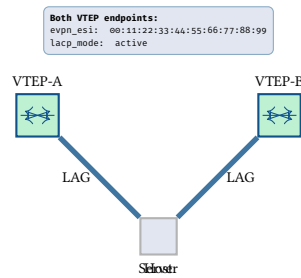


Figure 18. EVPN multi-homing with ESI LAG.

Table 39. Key attributes for EVPN multi-homing.

Attribute	Value	Notes
evpn_es1	00:11:22:33:44:55:66:77:88:99	Same on both VTEPs' server-facing endpoints
evpn_es1_mode	type-1	Manual ESI assignment
lacp_mode	active	Both sides run LACP
lacp_rate	fast	1-second LACP timeout for fast failover

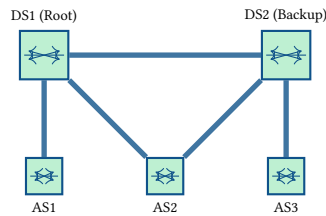


Figure 19. STP root placement with primary and secondary root bridges.

Table 40. Key attributes for STP root placement.

Attribute	DS1	DS2	AS*	Notes
stp_enabled	true	true	true	All switches
stp_mode	rpvst	rpvst	rpvst	Rapid PVST+
stp_priority	4096	8192	32768	Lowest wins root
stp_portfast	—	—	true	Access ports only
stp_bpdu_guard	—	—	true	Protect access ports

5.5.2 VRRP Active-Standby Pair

Description. Two routers share a virtual IP address on a LAN segment. One is active (higher priority), the other is standby. Hosts use the virtual IP as their default gateway.

When to use. Campus/enterprise networks requiring first-hop redundancy.

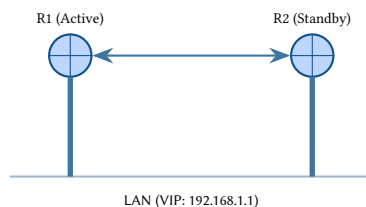


Figure 20. VRRP active-standby pair.

5.5.3 802.1X with Dynamic VLAN Assignment

Description. Access ports run 802.1X authentication. The RADIUS server returns a VLAN attribute, dynamically placing the host in the correct VLAN. Unauthenticated hosts fall into the guest VLAN.

When to use. Enterprise campus access layer with identity-based network access.

The authentication exchange involves three parties: the supplicant (end host), the access switch acting as the 802.1X authenticator, and the RADIUS server that holds the policy database. On successful authentication, the RADIUS server returns a VLAN attribute that the switch uses to dynamically assign the port. Failed or timed-out authentication falls back to the guest VLAN. The functor records these VLAN assignments in the L2

Table 41. Key attributes for VRRP active-standby.

Attribute	R1 (Active)	R2 (Standby)	Notes
vrrp_enabled	true	true	Both routers
vrrp_group	1	1	Same group
vrrp_virtual_ip	192.168.1.1	192.168.1.1	Shared VIP
vrrp_priority	150	100	Higher = active
vrrp_preempt	true	true	Preempt enabled
vrrp_version	3	3	VRRPv3

overlay so that downstream validation can verify VLAN consistency across the access layer. Figure 21 traces the message sequence.

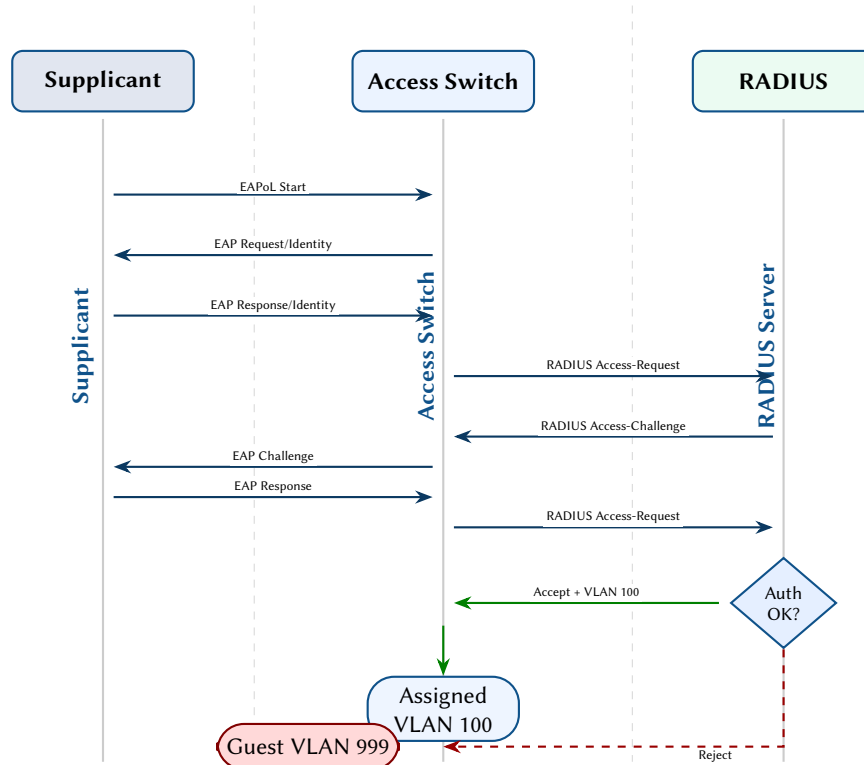


Figure 21. 802.1X authentication with dynamic VLAN assignment. The supplicant, access switch, and RADIUS server exchange EAPoL and RADIUS messages. On successful authentication, the switch assigns the port to the RADIUS-specified VLAN; on failure, the port falls to the guest VLAN.

Table 42. Key attributes for 802.1X with dynamic VLAN.

Attribute	Value	Notes
dot1x_enabled	true	All access ports
dot1x_mode	multi-auth	Multiple authenticated hosts per port
dot1x_guest_vlan	999	VLAN for unauthenticated hosts
dot1x_auth_fail_vlan	998	VLAN for auth failures
dot1x_mab	true	MAC bypass for non-802.1X devices
dot1x_order	dot1x-mab	Try 802.1X first, then MAB
switchport	access	Access mode
stp_portfast	true	Fast transition for edge ports
stp_bpdu_guard	true	Protection against rogue switches

The VLAN assignment attribute (`access_vlan`) is set dynamically by the RADIUS response; in the plan topology, it can be left unset or set to the guest VLAN as a static default. The functor records the `dot1x_guest_vlan` as the fallback VLAN for the L2 overlay.

5.6 Security Patterns

5.6.1 Hybrid-Cloud Security Normalization

Description. A unified security policy fragment that bridges on-premise SGTs with cloud-native security groups and Kubernetes labels. The same group ID (e.g., 100) is used to identify the group regardless of the underlying infrastructure.

When to use. Hybrid-cloud architectures where security policies must be enforced consistently across physical switches, public cloud VPCs, and container clusters.

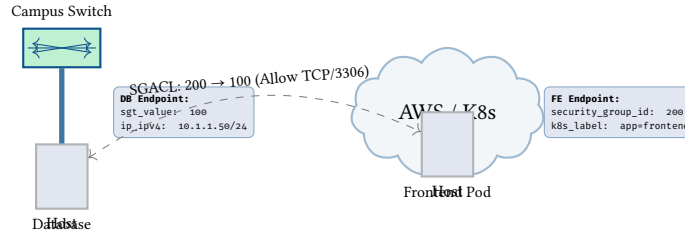


Figure 22. Hybrid-cloud security normalization.

Table 43. Key attributes for hybrid-cloud security.

Attribute	Value	Notes
sgt_value	100	Mapped to Cisco SGT on-prem
security_group_id	200	Mapped to AWS Security Group in cloud
k8s_label	app=frontend	Mapped to K8s NetworkPolicy selector
sgacl_src_sgt	200	Source of the security flow
sgacl_dst_sgt	100	Destination of the security flow
sgacl_protocol	tcp	SQL traffic match
sgacl_dst_port	3306	MySQL port match

Functor output. $\mathcal{F}_{\text{Security}}$ produces a compact security overlay graph G_{sec} where nodes are universal group identifiers. Renderers then translate these IDs into SGTs for IOS-XE, SG rules for AWS APIs, and YAML for Kubernetes.

5.7 Composite Patterns

5.7.1 DC Leaf-Spine with EVPN-VXLAN + eBGP Underlay

Description. The most common modern data-centre fabric design. eBGP provides the IP underlay (§5.1.4), while EVPN-VXLAN provides the overlay (§5.4.1). Each leaf runs as a VTEP with symmetric IRB.

Composition. On each node, the following attribute families are simultaneously present:

- bgp_* — eBGP underlay (unique ASN per device, L2VPN EVPN address family activated)
- evpn_* — EVPN control plane (VNIs, anycast gateway)
- ip_* — Loopback and P2P addressing

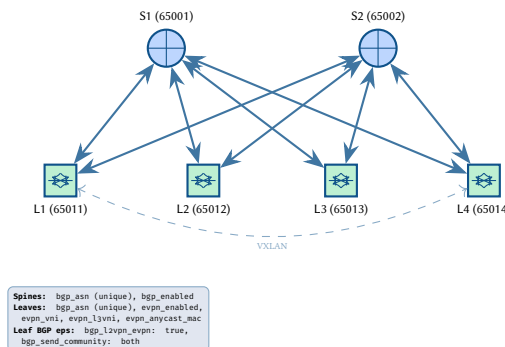


Figure 23. Composite: DC leaf-spine with eBGP underlay and EVPN-VXLAN overlay.

Functor cascade. $\mathcal{F}_{\text{BGP}}$ runs first to build the eBGP underlay overlay. $\mathcal{F}_{\text{EVPN}}$ then reads the BGP overlay to identify which VTEPs have L2VPN EVPN sessions and constructs the VXLAN tunnel overlay on top.

5.7.2 Enterprise WAN with MPLS + iBGP RR

Description. A service-provider or enterprise WAN with an MPLS core running LDP or SR, iBGP with route reflectors for service routes, and OSPF/IS-IS as the IGP underlay.

Composition. On each PE and P router, the following attribute families coexist:

- `ospf_*` or `isis_*` – IGP underlay
- `mpls_*` / `sr_*` – Label switching
- `bgp_*` – iBGP with RR for VPNv4/v6 routes

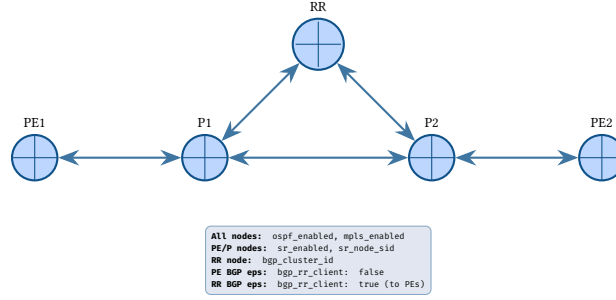


Figure 24. Composite: Enterprise WAN with MPLS + iBGP RR.

Functor cascade. $\mathcal{F}_{\text{OSPF}}$ (or $\mathcal{F}_{\text{ISIS}}$) builds the IGP overlay. $\mathcal{F}_{\text{MPLS}}$ reads the IGP overlay and SR/LDP attributes to construct label-switched paths. $\mathcal{F}_{\text{BGP}}$ builds the iBGP RR overlay for VPN service route distribution. The three overlays share the same underlying G_{plan} nodes but produce distinct edge sets and per-layer attributes.

6 Validation Rule Reference

The validation pipeline checks G_{plan} before any functor executes. Rules are organised into four categories: structural constraints, per-protocol consistency, cross-protocol consistency, and completeness. Each rule has a severity level:

- **Error** – the plan topology is invalid; functor execution is blocked until the error is resolved.
- **Warning** – the plan topology is technically valid but likely contains a configuration mistake; functor execution proceeds but the issue is reported.

The four categories execute in a fixed order of increasing cost, as shown in Figure 26. Structural checks ($O(N + E)$) run first because they are cheapest and catch gross errors early. Per-protocol checks then validate each layer independently. Cross-protocol checks examine interactions between layers (e.g., EVPN requires BGP). Completeness checks run last, verifying that every enabled protocol has all required attributes. Figure 25 provides a taxonomy of representative rules.

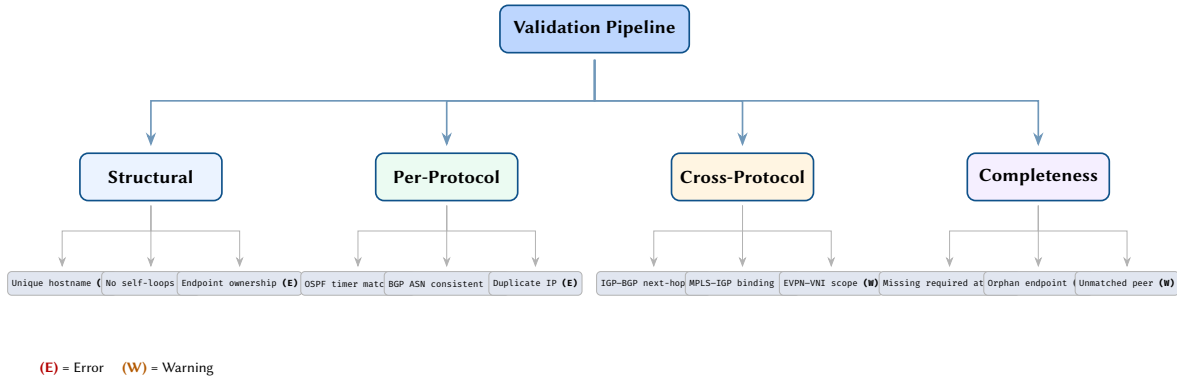


Figure 25. Taxonomy of validation rules. Each rule is tagged with its severity level: **(E)** errors block functor execution, **(W)** warnings are reported but execution proceeds. Representative rules are shown; the full set is defined in §6.

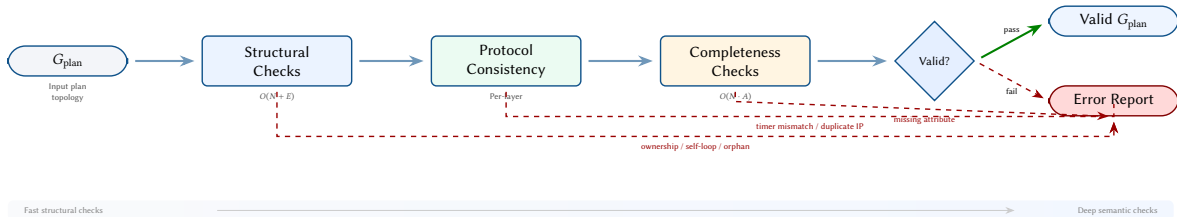


Figure 26. Validation pipeline for G_{plan} . The three stages execute in order of increasing cost: structural checks verify graph invariants in $O(N + E)$, protocol consistency validates per-layer constraints on connected endpoints, and completeness checks ensure all required attributes are present ($O(N \cdot A)$). Any stage may emit errors that halt the pipeline and produce a diagnostic report.

6.1 Structural Constraints

Design Rule 2: Unique Hostname

Every node $v \in V$ must have a unique hostname:

$$\forall v_i, v_j \in V, i \neq j \implies \pi_V(v_i)[\text{hostname}] \neq \pi_V(v_j)[\text{hostname}]$$

Severity: Error.

Design Rule 3: Unique Endpoint Name per Node

Within each node v , every endpoint $e \in \mathcal{E}(v)$ must have a unique name:

$$\forall e_i, e_j \in \mathcal{E}(v), i \neq j \implies \pi_E(e_i)[\text{name}] \neq \pi_E(e_j)[\text{name}]$$

Severity: Error.

Design Rule 4: Edge Endpoint Pairing

Every edge $(u, v) \in E$ must connect exactly one endpoint from $\mathcal{E}(u)$ to exactly one endpoint from $\mathcal{E}(v)$. An endpoint may participate in at most one edge (no multi-homing at the physical layer; LAGs are modelled as bundled logical endpoints).

Severity: Error.

Design Rule 5: IP Subnet Consistency

For every edge $(u, v) \in E$ connecting endpoints e_u and e_v with IPv4 addresses, the two addresses must be in the same subnet:

$$\text{subnet}(\pi_E(e_u)[\text{ip_ipv4}]) = \text{subnet}(\pi_E(e_v)[\text{ip_ipv4}])$$

Severity: Error.

Design Rule 6: No Duplicate IP Addresses

No two endpoints in G_{plan} may share the same IP address unless they are part of an anycast group (e.g., EVPN anycast gateway):

$$\forall e_i, e_j, i \neq j \implies \pi_E(e_i)[\text{ip_ipv4}] \neq \pi_E(e_j)[\text{ip_ipv4}]$$

Anycast addresses (evpn_anycast_ip, vrrp_virtual_ip) are exempt from this rule.

Severity: Error.

Design Rule 7: MTU Consistency

Both endpoints of an edge should have the same mtu value. MTU mismatches cause silent packet drops on jumbo-frame traffic.

Severity: Warning.

Structural checks are the cheapest validation stage ($O(N + E)$) and catch graph-level invariant violations before any protocol-specific analysis begins.

6.2 Per-Protocol Consistency Rules

Design Rule 8: OSPF Hello/Dead Interval Match

For every edge with OSPF-enabled endpoints, the hello and dead intervals must match:

$$\pi_E(e_u)[\text{ospf_hello}] = \pi_E(e_v)[\text{ospf_hello}] \quad \text{and} \quad \pi_E(e_u)[\text{ospf_dead}] = \pi_E(e_v)[\text{ospf_dead}]$$

Severity: Error.

Design Rule 9: OSPF Area Match

Both endpoints of an OSPF-enabled edge must belong to the same area:

$$\pi_E(e_u)[ospf_area] = \pi_E(e_v)[ospf_area]$$

Severity: Error.

Design Rule 10: OSPF Area Type Match

Both endpoints of an edge within the same OSPF area must agree on the area type (stub, NSSA, etc.):

$$\pi_E(e_u)[ospf_area_type] = \pi_E(e_v)[ospf_area_type]$$

Severity: Error.

Design Rule 11: OSPF Network Type Match

Both endpoints of an OSPF-enabled edge must use the same network type:

$$\pi_E(e_u)[ospf_network_type] = \pi_E(e_v)[ospf_network_type]$$

Severity: Error.

Design Rule 12: OSPF Authentication Match

Both endpoints must agree on authentication type and key:

$$\pi_E(e_u)[ospf_authentication] = \pi_E(e_v)[ospf_authentication]$$

If authentication is enabled, both sides must have `ospf_auth_key` set.

Severity: Error.

Design Rule 13: OSPF Dead = 4x Hello

On broadcast and NBMA networks, the dead interval should be four times the hello interval:

$$\pi_E(e)[ospf_dead] = 4 \times \pi_E(e)[ospf_hello]$$

Severity: Warning.

Design Rule 14: BGP Timer Agreement

Both endpoints of a BGP session should have matching timer values. Mismatched hold times cause the session to negotiate to the lower value, which may be unexpected.

Severity: Warning.

Design Rule 15: BGP Password Symmetry

If one side of a BGP session has `bgp_password` set, the other side must also have it set with the same value:

$$\pi_E(e_u)[bgp_password] = \pi_E(e_v)[bgp_password]$$

Severity: Error.

Design Rule 16: BGP Route Reflector Requires Cluster ID

If any endpoint on a node has `bgp_rr_client=true`, the node must have `bgp_cluster_id` set.

Severity: Error.

Design Rule 17: IS-IS Circuit Type Compatibility

An endpoint's `isis_circuit_type` must be compatible with the node's `isis_level`. An L1-only node cannot have L2 circuits.

Severity: Error.

Design Rule 18: IS-IS Metric Range

When `isis_metric_style=narrow`, the `isis_metric` must be in the range 1–63. When `isis_metric_style=wide`, the range is 1–16777215.

Severity: Error.

Design Rule 19: STP Priority Multiple

`stp_priority` must be a multiple of 4096. `stp_port_priority` must be a multiple of 16.

Severity: Error.

Design Rule 20: STP PortFast on Trunk Warning

If `stp_portfast=true` on an endpoint with `switchport=trunk`, a warning is raised. PortFast on trunk ports can cause spanning-tree loops.

Severity: Warning.

Design Rule 21: VRRP/HSRP Group Consistency

All endpoints in the same broadcast domain with the same VRRP/HSRP group ID must agree on the virtual IP address.

Severity: Error.

Design Rule 22: EVPN ESI Consistency

If two endpoints on different nodes share the same `evpn_es`, they must both have `evpn_enabled=true` on their parent nodes and agree on `evpn_esi_mode`.

Severity: Error.

Design Rule 23: MACsec Cipher Agreement

Both endpoints of a MACsec-protected link must use the same cipher suite and key name:

$$\pi_E(e_u)[\text{macsec_cipher}] = \pi_E(e_v)[\text{macsec_cipher}]$$

Severity: Error.

Design Rule 24: Auto-Negotiation Agreement

Both endpoints of a physical link must agree on `phy_auto_negotiation`:

$$\pi_E(e_u)[\text{phy_auto_negotiation}] = \pi_E(e_v)[\text{phy_auto_negotiation}]$$

Severity: Error.

Design Rule 25: LACP Mode Compatibility

If both endpoints of a LAG member link have `lacp_mode=passive`, the LAG will not form. At least one side must be active.

Severity: Error.

Per-protocol rules validate each layer in isolation, checking that paired endpoints agree on timer values, authentication settings, and protocol-specific identifiers. These checks run per-layer so that errors can be reported with clear protocol context.

6.3 Cross-Protocol Consistency Rules

Design Rule 26: EVPN Requires BGP L2VPN EVPN

If `evpn_enabled=true` on a node, then `bgp_enabled` must also be true, and at least one BGP endpoint must have `bgp_l2vpn_evpn=true`.

Severity: Error.

Design Rule 27: SR Requires IGP

If `sr_enabled=true`, at least one IGP must be active: `ospf_enabled=true` or `isis_enabled=true`. SR advertises its SIDs via IGP extensions.

Severity: Error.

Design Rule 28: LDP Requires MPLS

If `mpls_ldp_enabled=true` (node or endpoint), then `mpls_enabled=true` must be set on the node.

Severity: Error.

Design Rule 29: RSVP-TE Requires MPLS

If `mpls_rsvp_enabled=true`, then `mpls_enabled=true` must be set on the node.

Severity: Error.

Design Rule 30: SRGB Non-Overlap with Dynamic Range

The SR Global Block [`sr_global_block_start`, `sr_global_block_end`] must not overlap with the dynamic label range [`mpls_label_range_start`, `mpls_label_range_end`].

Severity: Error.

Design Rule 31: Symmetric IRB Requires L3 VNI

If `evpn_irb_mode=symmetric`, every EVPN-enabled endpoint must have `evpn_l3vni` set.

Severity: Error.

Design Rule 32: VXLAN Requires Underlay Reachability

EVPN-VXLAN VTEPs must have loopback reachability provided by an underlay IGP or eBGP. If `evpn_enabled=true`, at least one of `ospf_enabled`, `isis_enabled`, or `bgp_enabled` must be true.

Severity: Error.

Design Rule 33: Mutual Exclusion: OSPF and IS-IS on Same Interface

Running both OSPF and IS-IS on the same endpoint is technically possible but almost always a misconfiguration: if both `ospf_area` and `isis_metric` are present on the same endpoint, a warning is raised.

Severity: Warning.

Design Rule 34: Mutual Exclusion: VRRP and HSRP

An endpoint must not have both `vrrp_enabled=true` and `hsrp_enabled=true`.

Severity: Error.

Cross-protocol rules are the most architecturally significant: they verify that the dependency relationships in Figure 4 are satisfied (e.g., SR requires an IGP, EVPN requires BGP).

6.4 Completeness Rules

Design Rule 35: Protocol Gate Completeness

If a protocol's gate attribute (e.g., `ospf_enabled`) is true, all required attributes for that protocol must be present on the node and its relevant endpoints. For OSPF, this means `ospf_router_id` on the node and `ospf_area` on at least one endpoint.

Severity: Error.

Design Rule 36: BGP Peer Completeness

Every endpoint with `bgp_peer_addr` set must also have `bgp_peer_asn` set. Conversely, if `bgp_peer_asn` is present, `bgp_peer_addr` must also be present.

Severity: Error.

Design Rule 37: ACL Reference Integrity

Every ACL name referenced by `acl_ingress` or `acl_egress` on an endpoint must correspond to at least one node-level ACL rule attribute (`acl_[name]_seq_*`).

Severity: Error.

Design Rule 38: QoS Policy Reference Integrity

Every policy-map name referenced by `qos_ingress_policy` or `qos_egress_policy` must be a defined policy object.

Severity: Warning (policies may be defined externally).

Design Rule 39: Node SID Uniqueness

Every `sr_node_sid` value must be unique across all nodes in the topology. Duplicate node SIDs cause forwarding loops:

$$\forall v_i, v_j \in V, i \neq j \implies \pi_V(v_i)[sr_node_sid] \neq \pi_V(v_j)[sr_node_sid]$$

Severity: Error.

7 Worked Examples

7.1 Small Enterprise Network

This example constructs a plan topology for a small enterprise network with three routers (R1, R2, R3) and two switches (SW1, SW2). The network runs OSPF for IGP, iBGP between the routers with R1 as a route reflector, and VRRP for first-hop redundancy on the LAN segment. This example instantiates the OSPF single area pattern (§5.2.1), the single route reflector pattern (§5.1.2), and the VRRP active-standby pattern (§5.5.2).

7.1.1 Topology Description

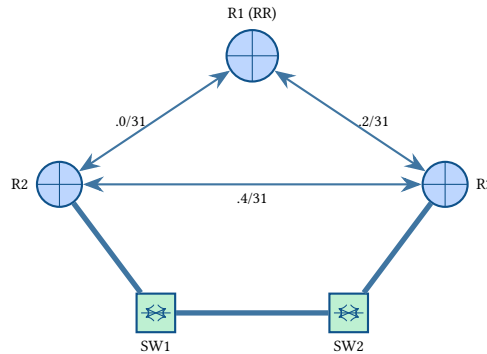


Figure 27. Small enterprise network: 3 routers, 2 switches.

7.1.2 Node Attributes

Table 44. Node attributes for the enterprise example.

Attribute	R1	R2	R3	SW1/SW2
hostname	core-rtr-01	dist-rtr-01	dist-rtr-02	acc-sw-01/02
role	core	distribution	distribution	access
ospf_enabled	true	true	true	—
ospf_router_id	10.255.0.1	10.255.0.2	10.255.0.3	—
ospf_reference_bandwidth	10000	10000	10000	—
bgp_enabled	true	true	true	—
bgp_asn	65000	65000	65000	—
bgp_router_id	10.255.0.1	10.255.0.2	10.255.0.3	—
bgp_cluster_id	10.255.0.1	—	—	—
stp_enabled	—	—	—	true
stp_mode	—	—	—	rpvst
stp_priority	—	—	—	32768

7.1.3 Endpoint Attributes – Router Interconnects

R1–R2 link (10.0.0.0/31):

- R1 endpoint (Ethernet1/1): ip_ipv4=10.0.0.0/31, ospf_area=0, ospf_network_type=point-to-point, bgp_peer_addr=10.255.0.2, bgp_peer_asn=65000, bgp_rr_client=true, bgp_update_source=Loopback0.
- R2 endpoint (Ethernet1/1): ip_ipv4=10.0.0.1/31, ospf_area=0, ospf_network_type=point-to-point, bgp_peer_addr=10.255.0.1, bgp_peer_asn=65000, bgp_update_source=Loopback0.

R1–R3 link (10.0.0.2/31):

- R1 endpoint (Ethernet1/2): ip_ipv4=10.0.0.2/31, ospf_area=0, ospf_network_type=point-to-point, bgp_peer_addr=10.255.0.3, bgp_peer_asn=65000, bgp_rr_client=true, bgp_update_source=Loopback0.
- R3 endpoint (Ethernet1/1): ip_ipv4=10.0.0.3/31, ospf_area=0, ospf_network_type=point-to-point, bgp_peer_addr=10.255.0.1, bgp_peer_asn=65000, bgp_update_source=Loopback0.

R2–R3 link (10.0.0.4/31):

- R2 endpoint (Ethernet1/2): ip_ipv4=10.0.0.4/31, ospf_area=0, ospf_network_type=point-to-point.
- R3 endpoint (Ethernet1/2): ip_ipv4=10.0.0.5/31, ospf_area=0, ospf_network_type=point-to-point.

7.1.4 Endpoint Attributes – VRRP and LAN

R2 LAN interface (Ethernet2/1):

- ip_ipv4=192.168.1.2/24, ospf_area=0, ospf_passive=true, vrrp_enabled=true, vrrp_group=1, vrrp_virtual_ip=192.168.1.1, vrrp_priority=150, vrrp_preempt=true.

R3 LAN interface (Ethernet2/1):

- ip_ipv4=192.168.1.3/24, ospf_area=0, ospf_passive=true, vrrp_enabled=true, vrrp_group=1, vrrp_virtual_ip=192.168.1.1, vrrp_priority=100, vrrp_preempt=true.

7.1.5 Endpoint Attributes – Switches

SW1 uplink to R2 (GigabitEthernet0/1):

- switchport=trunk, allowed_vlans="100,200", stp_port_cost=auto, lldp_enabled=true.

SW1 to SW2 inter-switch link (GigabitEthernet0/2):

- switchport=trunk, allowed_vlans="100,200", stp_guard=root.

SW1 access ports (GigabitEthernet0/3–0/24):

- switchport=access, access_vlan=100, stp_portfast=true, stp_bpdu_guard=true, dot1x_enabled=true, dot1x_mode=multi-auth, dot1x_guest_vlan=999.

7.1.6 Validation Results

Running the validation pipeline on this topology produces:

- All structural constraints pass (unique hostnames, unique endpoint names, subnet consistency).
- OSPF consistency checks pass (hello/dead match defaults, areas match on both sides of each link).
- BGP consistency checks pass (password not set on either side, cluster ID present on R1 which has RR clients).
- VRRP consistency passes (same virtual IP on R2 and R3 for group 1, VIP is in the 192.168.1.0/24 subnet).
- No cross-protocol issues (OSPF and BGP both enabled on routers, no conflicting configurations).

7.2 DC Leaf-Spine Fabric

This example constructs the plan topology for a data-centre leaf-spine fabric with 2 spines (S1, S2) and 4 leaves (L1–L4), running EVPN-VXLAN with an eBGP underlay. It composes the eBGP underlay pattern (§5.1.4), the symmetric IRB pattern (§5.4.1), and the DC leaf-spine composite pattern (§5.7.1).

7.2.1 Topology Description

The fabric uses the eBGP Clos underlay pattern (§5.1.4) with the symmetric IRB EVPN-VXLAN overlay (§5.4.1). Each device has a unique ASN. Leaves serve as VTEPs.

7.2.2 Node Attributes

7.2.3 Spine-to-Leaf Endpoint Attributes

Each spine-to-leaf link uses /31 point-to-point addressing. The following table shows S1's four downlinks (S2 follows the same pattern with addresses in the 10.0.2.0/24 block):

Table 45. Node attributes for the DC leaf-spine example.

Attribute	S1	S2	L1	L2	L3	L4
hostname	spine-01	spine-02	leaf-01	leaf-02	leaf-03	leaf-04
role	spine	spine	leaf	leaf	leaf	leaf
bgp_enabled	true	true	true	true	true	true
bgp_asn	65001	65002	65011	65012	65013	65014
bgp_router_id	10.0.0.1	10.0.0.2	10.0.1.1	10.0.1.2	10.0.1.3	10.0.1.4
bgp_bestpath_as_path	multipath-relax (all nodes)					
evpn_enabled	—	—	true	true	true	true
evpn_irb_mode	—	—	symmetric	symmetric	symmetric	symmetric
evpn_anycast_mac	—	—	00:00:5e:00:01:01 (all leaves)			

Table 46. S1 downlink endpoint attributes.

Attribute	S1-L1	S1-L2	S1-L3	S1-L4
name	Ethernet1/1	Ethernet1/2	Ethernet1/3	Ethernet1/4
ip_ipv4	10.0.1.0/31	10.0.1.2/31	10.0.1.4/31	10.0.1.6/31
bgp_peer_addr	10.0.1.1	10.0.1.3	10.0.1.5	10.0.1.7
bgp_peer_asn	65011	65012	65013	65014
bgp_ipv4_unicast	true	true	true	true
bgp_l2vpn_evpn	true	true	true	true
bgp_send_community	both	both	both	both
mtu	9216	9216	9216	9216

7.2.4 Leaf VTEP Endpoint Attributes

Each leaf has a loopback for the VTEP source IP and server-facing access/trunk ports.

Leaf loopback (Loopback0):

- `ip_ipv4=10.0.1.[1-4]/32` (unique per leaf).

Leaf VXLAN interface (nve1 or similar):

- `evpn_vni=10010` (VLAN 100 L2 VNI), `evpn_l3vni=10099` (L3 VNI for VRF “tenant-1”), `evpn_rd=auto`, `evpn_rt_import=auto`, `evpn_rt_export=auto`, `evpn_anycast_ip=10.1.100.254`, `evpn_arp_suppress=true`.

Leaf server-facing ports (Ethernet3/1–3/12):

- `switchport=access`, `access_vlan=100`, `stp_portfast=true`, `stp_bpdu_guard=true`, `mtu=9216`.

7.2.5 Functor Output Summary

1. $\mathcal{F}_{\text{BGP}}$: produces a bipartite eBGP overlay (spines $\leftrightarrow$ leaves) with 8 edges (2 spines $\times$ 4 leaves). Each edge carries the L2VPN EVPN and IPv4 unicast address families.
2. $\mathcal{F}_{\text{EVPN}}$: reads the BGP overlay and EVPN attributes to construct VXLAN tunnel edges between all leaf pairs. Produces $\binom{4}{2} = 6$ bidirectional tunnel edges for VNI 10010. The L3 VNI 10099 creates an additional set of tunnel edges for inter-subnet routing.
3. $\mathcal{F}_{\text{L2}}$ (STP functor): runs on the server-facing ports to model the access-layer L2 domain within each leaf. Since leaves are independent L2 domains (no inter-leaf STP), each leaf’s access ports form an isolated STP instance.

7.2.6 Validation Results

- Structural: all hostnames unique, all endpoint names unique per node, IP subnets consistent on /31 links, no duplicate IPs.
- BGP: peer ASNs match the remote device’s `bgp_asn`, `bgp_send_community=both` on all EVPN sessions (required for extended communities), `bgp_l2vpn_evpn=true` on all spine-leaf sessions.
- EVPN: `evpn_l3vni` present on all leaves (required by symmetric IRB), `evpn_anycast_mac` identical across all leaves, BGP L2VPN EVPN address family activated.
- Cross-protocol: EVPN requires BGP — satisfied. No IGP is used (eBGP underlay provides reachability) — the VXLAN underlay check passes because `bgp_enabled=true`.
- MTU: all inter-switch links use MTU 9216 consistently.

8 Migration Guide

This section provides guidance on populating G_{plan} from existing device configurations. Migrating from vendor-specific CLI to the flat attribute schema is the most common onboarding task for new users of the plan topology. The process is iterative: initial mappings often surface inconsistencies that the validation pipeline catches, requiring corrections before the plan topology is ready for functor execution. Figure 28 shows the end-to-end workflow with its feedback loop.

8.1 Populating G_{plan} from Existing Configs

The migration workflow has four steps:

1. **Inventory.** Enumerate all devices and their interconnections. Each device becomes a node in G_{plan} ; each physical interface becomes an endpoint; each cable becomes an edge.
2. **Extract.** For each device, parse the running configuration and extract the relevant stanzas (interface blocks, router process blocks, ACL definitions, etc.).
3. **Map.** Convert each configuration stanza to flat attributes using the vendor-specific mappings below.
4. **Validate.** Run the validation pipeline (§6) to catch inconsistencies introduced during migration.

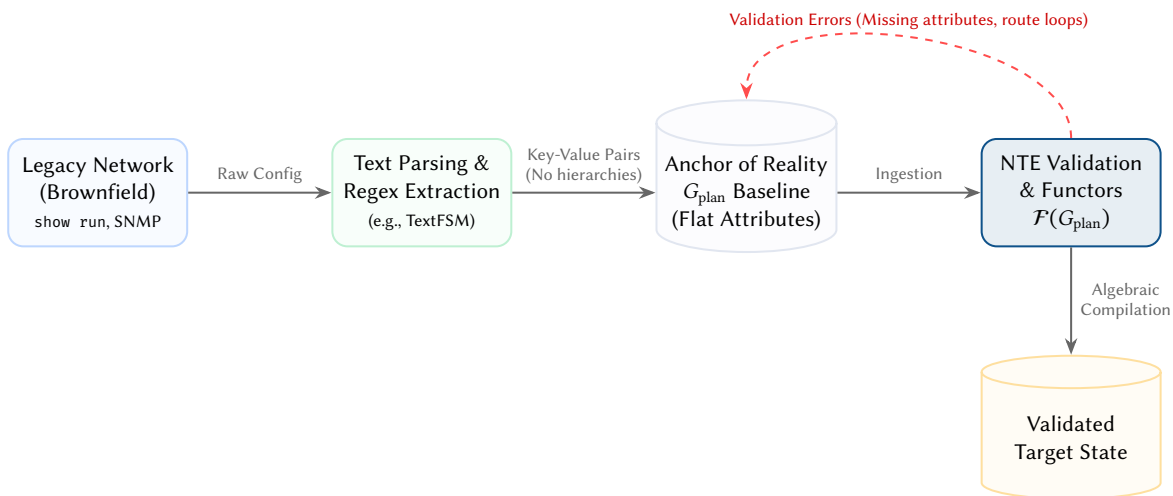


Figure 28. Brownfield Ingestion Workflow: Because the Plan Topology (G_{plan}) relies on flat, columnar attributes rather than deep YAML hierarchies, ingesting legacy configurations requires only basic regex key-value extraction. The mathematical burden of validating the state and building the relational links is offloaded entirely to the NTE Functors.

8.2 Cisco IOS/IOS-XE Mapping

8.3 Juniper Junos Mapping

8.4 Arista EOS Mapping

8.5 Handling Vendor-Specific Features

Not every vendor CLI knob maps to the plan topology schema. The following guidelines address common edge cases:

1. **Vendor-specific optimisations** (e.g., Cisco `ip cef`, Arista hardware `tcam`): these are platform-specific implementation details, not protocol state. They are not represented in G_{plan} . Document them in device-level metadata outside the attribute schema.
2. **Deprecated or aliased commands** (e.g., IOS `spanning-tree portfast trunk` vs. `spanning-tree edge-port`): map to the canonical attribute name. In this case, both map to `stp_portfast=true`.
3. **Features not yet in the schema** (e.g., PIM multicast, NAT, PBR): use the extension namespace `x_<vendor>_<feature>_*` as a temporary measure. For example, `x_cisco_pim_mode=sparse-mode`. These attributes are ignored by the standard functors but preserved in the topology for future schema extensions.
4. **Implicit defaults**: many vendors omit configuration lines for default values (e.g., OSPF hello of 10 seconds is never shown in the running config). When migrating, either leave the attribute absent (relying on the schema default from §3) or explicitly set it for clarity. The validation pipeline treats absent optional attributes as carrying their documented default values.
5. **Context-dependent configuration**: some commands change meaning based on context (e.g., `network under router ospf` vs. `router bgp`). The flat attribute schema eliminates this ambiguity by using protocol-prefixed keys.

Table 47. Cisco IOS CLI to plan topology attribute mapping.

IOS CLI	Plan Attribute	Notes
hostname X	hostname=X	Node-level
interface Gi0/1	name=GigabitEthernet0/1	Endpoint identity
ip address A M	ip_ipv4=A/len	Convert mask to prefix length
ip ospf cost C	ospf_cost=C	Endpoint-level
ip ospf hello-interval H	ospf_hello=H	Endpoint-level
ip ospf dead-interval D	ospf_dead=D	Endpoint-level
ip ospf network point-to-point	ospf_network_type=point-to-point	
ip ospf area A	ospf_area=A	Note: can be dotted decimal
router ospf P	ospf_process_id=P, ospf_enabled=true	Node-level
router-id R	ospf_router_id=R	Under router ospf
router bgp A	bgp_asn=A, bgp_enabled=true	Node-level
neighbor X remote-as Y	bgp_peer_addr=X, bgp_peer_asn=Y	Endpoint-level
neighbor X route-reflector-client	bgp_rr_client=true	On the endpoint facing X
neighbor X update-source L	bgp_update_source=L	
neighbor X send-community both	bgp_send_community=both	
switchport mode access	switchport=access	
switchport access vlan V	access_vlan=V	
switchport trunk allowed vlan V1,V2	allowed_vlans="V1,V2"	
spanning-tree vlan V priority P	stp_priority=P	Per-VLAN on node
spanning-tree portfast	stp_portfast=true	Endpoint-level

Table 48. Junos configuration to plan topology attribute mapping.

Junos Config Path	Plan Attribute	Notes
system host-name X	hostname=X	Direct pre- fix no- ta- tion
interfaces xe-0/0/0 unit 0 family inet address A/L	ip_ipv4=A/L	
protocols ospf area A interface xe-0/0/0	ospf_area=A	
protocols ospf area A interface xe-0/0/0 metric M	ospf_cost=M	Area in hi- er- ar- chy, in- ter- face as end- point
protocols ospf area A interface xe-0/0/0 interface-type p2p	ospf_network_type=point-to-point	
protocols ospf area A interface xe-0/0/0 passive	ospf_passive=true	
protocols bgp group X type internal	bgp_peer_type=ibgp	
protocols bgp group X neighbor A	bgp_peer_addr=A	Junos uses group- level type Per- neighbour Cluster im- plies RR client
protocols bgp group X cluster A	bgp_rr_client=true	
routing-options autonomous-system A	bgp_asn=A	
protocols isis interface xe-0/0/0 level 2 metric M	isis_metric=M, isis_circuit_type=L2	Node- level
protocols mpls interface xe-0/0/0	mpls_ldp_enabled=true	

Table 49. Arista EOS CLI to plan topology attribute mapping.

EOS CLI	Plan Attribute	Notes
hostname X	hostname=X	Same as IOS
interface Ethernet1	name=Ethernet1	EOS uses short names
ip address A/L	ip_ipv4=A/L	Direct prefix notation
router bgp A neighbor X remote-as Y address-family evpn / neighbor X activate	bgp_asn=A, bgp_enabled=true bgp_peer_addr=X, bgp_peer_asn=Y bgp_l2vpn_evpn=true	On end-point facing X
vxlan interface Vxlan1 / vxlan vni V vxlan interface Vxlan1 / vxlan source-interface Loo ip virtual-router mac-address M	evpn_vni=V VTEP source = Loopback0 IP evpn_anycast_mac=M	Node-level
ip virtual-router address A	evpn_anycast_ip=A	Per-SVI/VLAN endpoint
interface Vlan100 / ip address virtual A/L	evpn_anycast_ip=A	EOS virtual address

References

- [1] S. Knight, “A vendor-neutral multi-layer graph model for network protocol state,” Tech. Rep. ANK-TR-2026-03, Mar. 2026.